

U R Rao Satellite Centre (URSC)
Indian Space Research Organisation (ISRO)
Mechanical Integration Division – 1, Systems Integration Group

Optimization of Spacecraft Configuration

Automated Dual-Sided Component Placement using
Differential Evolution, CMA-ES and Deep Reinforcement Learning (PPO)

Internship Project Report

Submitted in partial fulfilment of the requirements of the ISRO-URSC Internship Programme

Submitted by:

Neeraj P Rao
B.Tech (Hons.) Computer Science and
Engineering
RV University, Bengaluru

Guided by:

Shravan Kumar Thakur
Prashant D Bagalkot

August 13, 2026

Certificate

This is to certify that the project report entitled “**Optimization of Spacecraft Configuration**” is a bonafide record of the internship work carried out by Neeraj P Rao, a student of RV University, Bengaluru.

The project was successfully executed at the Mechanical Integration Division – 1 (MID-1), Systems Integration Group (SIG), located at the U R Rao Satellite Centre (URSC), Indian Space Research Organisation (ISRO), Bengaluru, under the esteemed guidance and supervision of Mr. Shravan Kumar Thakur and Mr. Prashant D Bagalkot. To the best of our knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any degree, diploma, or title.

Guide 1: Shravan Kumar Thakur

Guide 2: Prashant D Bagalkot

Acknowledgements

I would like to express my sincere gratitude to the U R Rao Satellite Centre (URSC), Indian Space Research Organisation (ISRO), especially the System Integration Department, for providing me with the opportunity to undertake this Project work in a highly professional and technically enriching environment. This experience has greatly contributed to my academic learning and practical understanding of Mechanical engineering and system integration.

I would like to express my respectful gratitude to Shri. SHREE NIWAS SAHU Sir, Group Director (SIG-System Integration Group, ICA-Integration Checkout Area), for his leadership and support during this Project Internship.

I also like to express my gratitude toward Shri. PRASHANT D BAGALKOT sir, Division Head (MID-1, SIG, ICA), Shri. SHRAVAN KUMAR THAKUR sir, (MIS-1, MID-1, SIG, ICA) for his Guidance and Support throughout my project trainee period.

Special thanks to TEJAS G and GOURAV S HAZARE, for his cooperation, support, and helpful nature during the Project Internship period.

Finally, I thank all the engineers, staff members, and colleagues at URSC, ISRO who directly or indirectly contributed to my learning experience.

I remain truly grateful for this opportunity, which has provided me with valuable technical exposure and practical experience.

Abstract

Manual placement of electronic and mechanical components on a spacecraft mounting panel is a slow, error-prone process that frequently results in an offset Centre of Gravity (CoG), which in turn creates attitude-control instabilities during flight. This project develops and compares three independent computational approaches to automate the placement of two-dimensional components across the front and back faces of a panel such that the combined assembly CoG lies as close as possible to the panel origin, while satisfying a hierarchy of hard physical constraints: zero overlap between component bodies (with direction-dependent clearance and special “Connector Face” clearances), containment within a border margin, and minimum pin-to-pin separation between structural insert holes (4 mm and 6 mm diameter, requiring 24–36 mm separation depending on the pairing). The three solvers implemented are Differential Evolution (DE), Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and Proximal Policy Optimization (PPO), a deep reinforcement-learning algorithm. All three were accelerated using Numba JIT compilation for the constraint-evaluation core and were engineered to output five distinct, fully compliant candidate layouts per run so that design engineers can select the alternative best suited to downstream requirements. A dedicated comparative study evaluates the three solvers on wall-clock computational cost, convergence speed, scalability with respect to the number of components, statistical significance of performance differences, solution diversity, multi-board amortized generation speed, and constraint-violation composition. The results show that while DE and CMA-ES are simple to configure and reliably constraint-compliant, they must re-solve the optimization problem from scratch for every new board, whereas a trained PPO policy amortizes its training cost and generates new, fully valid layouts in a fraction of a second through direct neural-network inference. The report also documents the full engineering pipeline – from the original guide-provided DE prototype, through progressively richer constraint modelling (border, clearance, insert holes), to the final multi-algorithm, front/back-aware placement system – and applies the completed pipeline to a new, previously unseen panel layout supplied by the project guide.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview of Indian Space Research Organisation (ISRO) & U R Rao Satellite Centre (URSC)

Established in 1969 and headquartered in Bengaluru, ISRO is India's national space agency, mandated to develop indigenous space technology for national development and scientific exploration. Operating through the PSLV and GSLV launch vehicles, ISRO deploys the GSAT, IRS, and NavIC satellite constellations for communication, Earth observation, navigation, and disaster management. Flagship missions such as Chandrayaan-1 and the Mars Orbiter Mission validated its technical capabilities, while a growing commercial launch portfolio and strategic focus on Gaganyaan, reusable launch vehicles, small launchers, deep-space missions, and international partnerships position India as a self-reliant spacefaring nation.

Insert the U R Rao Satellite Centre image here.

Save the image as `URSC.png` in the Overleaf project folder.

Figure 1.1: U R Rao Satellite Centre, Bengaluru, Karnataka

The U R Rao Satellite Centre (URSC) in Bengaluru is ISRO's primary hub for the end-to-end design, integration, and qualification of satellites. URSC executes the complete development lifecycle—from conceptual design and structural-thermal analysis to payload integration and environmental testing in thermal-vacuum, vibration, and acoustic facilities. The centre delivers flight-ready platforms across communication, Earth observation, navigation, and scientific domains, with core expertise in power systems, attitude and orbit control, and advanced communication payloads. URSC remains the operational backbone for ISRO's next-generation

missions, including human spaceflight, deep-space exploration, and high-throughput satellite constellations.

1.2 Background

Spacecraft subsystems are built up from printed circuit boards (PCBs) and mechanical panels onto which dozens of discrete components – power distribution units, sensors, connectors, radiating plates, mounting brackets, and structural inserts – must be rigidly fixed. Because a spacecraft is a free body in orbit, any imbalance in how mass is distributed across its structure translates directly into a Centre of Gravity (CoG) offset from the intended geometric centre. During launch and orbital manoeuvres, a CoG offset produces unwanted torques that the Attitude and Orbit Control System (AOCS) must continuously correct for, consuming propellant and, in extreme cases, threatening mission stability. Component placement is therefore not a purely aesthetic or space-utilisation problem; it is a mass-balance engineering problem with direct consequences for spacecraft dynamics.

Historically, this placement task has been carried out manually by mechanical design engineers using CAD tools, iterating by trial and error until an acceptable CoG offset and a collision-free, manufacturable layout are reached. This process does not scale well: as the number of components grows, or as new board geometries are introduced, the manual search space grows combinatorially, and engineers have no systematic way of exploring multiple good alternatives beyond the first one they happen to find.

This project, carried out at the Mechanical Integration Division – 1 of the Systems Integration Group, U R Rao Satellite Centre (ISRO), automates this placement task using numerical and machine-learning-based optimization. The work began from a simple Differential-Evolution-based prototype supplied by the project guide and was progressively extended – first by adding a rigorous constraint model (board border, inter-component clearance, connector-face clearance, and insert-hole proximity rules), then by generalising to dual-sided (front and back) panels, and finally by implementing and comparing three qualitatively different optimization paradigms: Differential Evolution, CMA-ES, and Proximal Policy Optimization.

1.3 Problem Statement

Given a rectangular mounting panel of specified width and height, and a set of components – each defined by its planar footprint (rectangular or circular), mass, insert-hole configuration, and the panel face (front or back) it must be mounted on – determine the (x, y) coordinates of every component such that:

- (i) The combined mass-weighted Centre of Gravity of the full assembly lies within a small tolerance of the panel’s geometric origin $(0, 0)$;
- (ii) No two components placed on the same face overlap, taking into account a minimum clearance envelope (which may be direction-dependent through a “Connector Face” override);
- (iii) Every component, including its clearance envelope, remains within the usable panel area after subtracting a fixed border margin;
- (iv) Every structural insert hole maintains the minimum required separation (24 mm, 30 mm, or 36 mm, depending on whether the paired inserts are 4 mm, mixed, or 6 mm in diameter) from every insert hole belonging to a different component, on both faces simultaneously since inserts pass through the full panel thickness;
- (v) Five distinct, fully compliant layouts are produced per run so that a design engineer has multiple valid alternatives to choose from.

1.4 Objectives

The project set out to achieve the following objectives:

- Develop a parallelised optimization tool that automates (x, y) component placement without manual iteration.
- Restrict the combined assembly CoG offset to as minimum as possible of the panel origin.
- Fully enforce all geometric and structural constraints: zero collisions between component bodies and clearance envelopes, panel border containment, and insert-to-insert minimum separation.
- Implement and benchmark multiple optimization paradigms – Differential Evolution, CMA-ES, and PPO – accelerated using Numba JIT compilation.
- Generate five distinct, fully compliant layout alternatives per optimization run.
- Quantitatively compare the three algorithms along axes relevant to a production engineering workflow: computational cost, convergence behaviour, scalability, statistical robustness, solution diversity, and amortized multi-board generation speed.

1.5 Scope and Constraints Overview

The scope of the project is restricted to planar (two-dimensional) component placement; component orientation/rotation, thermal considerations, and out-of-plane structural stresses are outside the scope of this work. Four categories of engineering constraint drive the optimization, summarised in Table ?? and elaborated fully in Chapter ??.

Table 1.1: Summary of engineering constraints modelled in the placement problem

Constraint	Description
Mass & Balance	Combined assembly CoG must lie within tolerance of the panel origin (0,0).
Dual-sided packing	Components are placed independently on FRONT and BACK panel faces; same-face components must not collide.
Keepout / clearance	Zero-overlap rule for component bodies plus a configurable clearance envelope, including a directional Connector-Face (CF) override.
Insert proximity	Insert holes (4.0 mm / 6.0 mm diameter) require 24–36 mm minimum pairwise separation depending on diameter combination, enforced across both faces.

1.6 Organization of the Report

Chapter ?? surveys the algorithmic and domain literature underpinning the three solvers. Chapter ?? formalises the placement problem mathematically, including the CoG definition and every constraint. Chapter ?? details the engineering methodology and the evolution of the pipeline from the original DE prototype through to the final multi-algorithm system. Chapter ?? describes implementation aspects: the JSON-driven configuration schema, the Numba-accelerated fitness core, and the visualization pipeline. Chapter ?? presents and discusses results for all three algorithms individually, the head-to-head comparative study, and a case study applying the completed pipeline to a new panel layout. Chapter ?? documents the engineering challenges encountered and how they were resolved. Chapters ?? and ?? present future scope and concluding remarks, followed by references and appendices containing selected code listings.

Chapter 2

Literature Review

2.1 Automated Component and PCB Placement

Automated placement of components on a substrate is a well-studied combinatorial optimization problem in the electronic design automation (EDA) community. Classical approaches such as the TimberWolf placement and routing package [?] used simulated-annealing-style stochastic search to place standard-cell circuits, establishing the template that later metaheuristic placement tools would follow: represent a layout as a vector of coordinates, define a scalar cost (penalty) function encoding all design constraints, and search the coordinate space to minimise that cost. More recently, machine-learning-driven placement has entered mainstream chip design; Mirhoseini et al. [?] demonstrated a graph-placement methodology using deep reinforcement learning that produced chip floorplans competitive with, and in some metrics superior to, those produced by human engineers, in a fraction of the time. This project draws directly on both traditions: the metaheuristic (DE, CMA-ES) side follows the classical penalty-function search paradigm, while the PPO solver follows the RL-driven placement paradigm popularised by Mirhoseini et al.

2.2 Differential Evolution

Differential Evolution (DE), introduced by Storn and Price [?], is a population-based, gradient-free global optimizer for continuous search spaces. At every generation, DE creates a trial vector for each population member by adding the weighted difference of two randomly selected population vectors to a third, then applies crossover with the current member and greedily accepts the trial if it improves the objective. DE is attractive for this project because it requires no gradient information (the placement penalty function is highly discontinuous due to hard overlap/border indicator terms), is embarrassingly parallel across the population, and has very few tunable hyperparameters (population size, mutation/crossover behaviour, and the maximum number of generations).

2.3 Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

CMA-ES [?] is a state-of-the-art evolution strategy for difficult, non-convex, non-separable continuous optimization problems. Unlike DE, CMA-ES explicitly maintains and adapts a full covariance matrix describing the search distribution, allowing the algorithm to learn correlations

between decision variables (e.g. that moving one component’s x and y coordinates together is more productive than moving them independently) and to dynamically rescale its step size. This adaptive behaviour makes CMA-ES particularly effective at escaping the local minima and narrow “geometric deadlocks” that a simpler evolutionary strategy such as DE can become trapped in when components are close to a valid configuration but cannot move without transiently increasing the overlap penalty.

2.4 Reinforcement Learning and Proximal Policy Optimization

Reinforcement learning (RL) frames a task as a Markov Decision Process (MDP) in which an agent iteratively observes a state, takes an action, and receives a reward, with the goal of learning a policy that maximises cumulative reward [?]. Proximal Policy Optimization (PPO) [?], introduced by Schulman et al. at OpenAI, is a model-free, on-policy, Actor-Critic policy-gradient algorithm that constrains each policy update using a clipped surrogate objective, preventing destructively large parameter updates while retaining much of the sample efficiency of more complex trust-region methods. PPO is widely regarded as an industry-standard baseline for continuous-control RL problems due to its favourable balance of implementation simplicity, training stability, and performance, which motivated its selection as the RL solver for this project. Critically, unlike DE and CMA-ES, a trained PPO policy is a reusable artefact: once trained, it can generate new valid layouts for previously unseen problem instances via a single forward pass, at a fraction of the wall-clock cost of re-running an evolutionary search from scratch.

2.5 High-Performance Computing Support

All three solvers rely on Numba [?], a just-in-time (JIT) compiler that translates a subset of Python and NumPy code directly into optimized machine code at first call, giving compiled-language performance for the innermost constraint-evaluation loops (pairwise overlap tests, insert-distance tests, and CoG computation) while retaining Python’s development ergonomics for the surrounding orchestration code. The PPO solver additionally relies on Stable-Baselines3 [?], a widely used, well-tested library of RL algorithm implementations built on PyTorch, and on the Gymnasium environment API for defining the placement MDP.

2.6 Summary

The literature establishes three complementary families of solution technique for the placement problem at hand: classical stochastic metaheuristics with minimal assumptions (DE), adaptive metaheuristics that learn problem structure within a single run (CMA-ES), and amortized, learning-based inference (PPO). This project is a novel application of all three techniques, side by side, to the specific problem of dual-sided spacecraft panel component placement under simultaneous CoG, clearance, and insert-hole proximity constraints.

Chapter 3

Problem Formulation

3.1 Centre of Gravity

The Centre of Gravity (CoG) of a system of objects is the point at which the entire weight of the system may be considered to act. For a set of N point masses m_i located at planar positions (x_i, y_i) , the CoG is

$$\bar{x} = \frac{\sum_{i=1}^N m_i x_i}{\sum_{i=1}^N m_i}, \quad \bar{y} = \frac{\sum_{i=1}^N m_i y_i}{\sum_{i=1}^N m_i}. \quad (3.1)$$

For a dual-sided panel, front-side and back-side components both contribute to a single, shared CoG. Since the back face is viewed as a mirror image of the front face along the panel's local x -axis, the x -coordinate of every back-side component is negated before being substituted into Equation ??, so that a component near the panel edge on the back face contributes correctly to the assembly's true CoG in the panel's physical reference frame. The y -coordinate and the overall two-dimensional balance point are unaffected by which face a component sits on. The design goal is

$$\sqrt{\bar{x}^2 + \bar{y}^2} \leq \text{CG_TOLERANCE}, \quad \text{CG_TOLERANCE} = 1.0 \text{ mm}. \quad (3.2)$$

3.2 Design Variables

For an assembly of N components, the optimization vector $\mathbf{v} \in \mathbb{R}^{2N}$ consists of the interleaved planar coordinates

$$\mathbf{v} = (x_1, y_1, x_2, y_2, \dots, x_N, y_N), \quad (3.3)$$

with each (x_i, y_i) representing the centre position of component i within its assigned panel face. Each component additionally carries static (non-optimized) attributes read from the JSON configuration file: object name, shape (rectangle or circle), length/breadth or diameter, mass, insert diameter, insert quantity, an optional Connector-Face (CF) override, and the face (FRONT/BACK) it is mounted on.

3.3 Constraint 1: Border Containment

The panel defines a usable placement region obtained by inseting the full panel rectangle by a fixed **Border** (mm) margin on all four sides. Every component's full footprint – including its clearance envelope – must lie entirely within this inset region. In the original guided prototype,

the panel width, height, and a 5-component test assembly were entered interactively (see Section ??); in the automated pipeline this is read directly from the JSON configuration, e.g. a 20 mm border margin on a 200×200 mm test panel.

3.4 Constraint 2: Clearance Space and Connector-Face Override

No two components mounted on the same panel face may overlap, and in addition a minimum clearance distance must separate their bodies. This clearance is direction-dependent: a component may declare a **CF** (Connector Face) list identifying which of its four sides face a wiring/connector interface, together with a **CFLen** (mm) value specifying a larger, connector-specific clearance to apply on that side (e.g. 65 mm in several test configurations), while ordinary clearance on the remaining sides uses the smaller board-level **Clearance** (mm) value. Clearance envelopes are permitted to overlap the border margin, but never another component's body.

The overlap penalty itself is computed axis-aligned using an interval-intersection formulation. For two components with bounding intervals $[a_1, a_2]$ and $[b_1, b_2]$ along one axis,

$$\text{overlap} = \max(0, \min(a_2, b_2) - \max(a_1, b_1)). \quad (3.4)$$

The worked numerical example used to validate this logic during development is:

$$\text{Overlap}_x = \max(0, \min(2, 3) - \max(0, 1)) = \max(0, 2 - 1) = 1 \quad (3.5)$$

$$\text{Overlap}_y = \max(0, \min(2, 3) - \max(0, 1)) = \max(0, 2 - 1) = 1 \quad (3.6)$$

$$\text{Overlap area} = 1 \times 1 = 1 \quad (3.7)$$

$$\text{Overlap penalty} = 1 \times 100 = 100 \quad (3.8)$$

i.e. the overlap area is multiplied by a large penalty weight (100 in this example) so that even a small residual overlap dominates the scalar objective and is aggressively driven to zero by the optimizer.

3.5 Constraint 3: Insert Hole Proximity

Each component may carry one or more structural insert holes, each either 4.0 mm or 6.0 mm in diameter, used to physically bolt the component to the panel. Because inserts belonging to different components can be placed arbitrarily close together in (x, y) even when the component bodies themselves satisfy clearance, a separate minimum-distance rule governs insert pairs, summarised in Table ??.

Table 3.1: Minimum required insert-to-insert separation by diameter pairing

Insert A (mm)	Insert B (mm)	Minimum separation (mm)
4.0	4.0	24.0
4.0	6.0	30.0
6.0	6.0	36.0

Because insert holes are physical through-holes that pass entirely through the panel thickness, this rule is enforced simultaneously across both the FRONT and BACK faces – an insert belonging to a front-side component and an insert belonging to a back-side component are still required to respect the minimum separation table above, even though the two components' bodies themselves are on opposite faces and therefore never collide.

3.6 Front versus Back Placement

A component's assigned face changes only its position along the panel's local depth (Z) axis; its (x, y) position is optimized identically regardless of face. The practical consequences, established early in the project and enforced throughout every solver, are:

- **Component clearance:** clearance/overlap rules apply only between components sharing the same face, since front and back components physically cannot collide.
- **Insert holes:** insert-to-insert distance rules apply across both faces simultaneously (Section ?? and above), since inserts are through-holes.
- **Centre of Gravity:** the CoG calculation is face-agnostic in the sense that every component's mass contributes to one shared 2D balance point, but a back-side component's x -coordinate must first be mirrored (negated) to express it in the panel's single physical reference frame, as described in Section 3.1.

3.7 JSON Configuration Schema

To avoid re-entering dozens of component specifications by hand for every experiment, the project's configuration is captured in a JSON file (e.g. `layout1.json`, `layout2.json`) with the schema illustrated in Listing ??, derived directly from the working configuration files used throughout the project.

```

1 {
2   "BOARD": {
3     "Object name": "BOARD",
4     "Length (mm)": 960.0,
5     "Breadth (mm)": 582.4,
6     "Weight (kg)": 2.0,
7     "Border (mm)": 20.0,
8     "Clearance (mm)": 20.0
9   },
10  "COMPONENTS": {
11    "FRONT": [
12      {
13        "Object name": "PDS",
14        "Length (mm)": 100.0,
15        "Breadth (mm)": 112.0,
16        "Weight (kg)": 1.5,
17        "Insert (mm)": 4.0,
18        "Shape": "rectangle",

```

```

19     "Insert qty": 4,
20     "Object qty": 1,
21     "CF": [1, 3],
22     "CFLen (mm)": 65.0
23 },
24 {
25     "Object name": "CCRR",
26     "Diameter (mm)": 45.0,
27     "Weight (kg)": 0.15,
28     "Insert (mm)": 6.0,
29     "Shape": "circle",
30     "Insert qty": 6,
31     "Object qty": 2,
32     "CFLen (mm)": 65.0
33 }
34 ],
35 "BACK": [
36     { "Object name": "BM1", "...": "..." }
37 ]
38 }
39 }

```

Listing 3.1: Excerpt of the JSON board/component configuration schema (from DE/layout1.json)

Each component entry declares its own geometry (rectangular **Length/Breadth** or circular **Diameter**), **Weight**, insert configuration (**Insert (mm)** diameter and **Insert qty**), an optional **Object qty** for repeated instances of the same physical part (expanded internally into individually indexed components, e.g. **CCRR0**, **CCRR1**), and an optional directional **CF/CFLen** override for connector-face clearance. This schema is shared, unmodified, by all three solvers (DE, CMA-ES, PPO), which is what allows their outputs to be compared on a like-for-like basis in Chapter ??.

3.8 Objective / Penalty Function Formulation

All three solvers reduce the multi-constraint placement problem to the minimisation of a single, non-negative scalar penalty function $J(\mathbf{v})$ evaluated on a candidate coordinate vector $\mathbf{v}$:

$$J(\mathbf{v}) = w_b \cdot P_{\text{border}}(\mathbf{v}) + w_o \cdot P_{\text{overlap}}(\mathbf{v}) + w_i \cdot P_{\text{insert}}(\mathbf{v}) + w_c \cdot P_{\text{cg}}(\mathbf{v}), \quad (3.9)$$

where P_{border} accumulates the squared out-of-bounds distance for any component that violates the border-inset region, P_{overlap} accumulates pairwise overlap area (Section 3.3) across all same-face component pairs, P_{insert} accumulates the squared shortfall below the required minimum separation for every insert pair violating Table ??, and P_{cg} penalises the squared CoG offset from the origin. A layout is considered *fully valid* only when $P_{\text{border}} = P_{\text{overlap}} = P_{\text{insert}} = 0$ exactly (the three “hard” priority-1 constraints), with P_{cg} minimised as the “soft” priority-2 objective once the hard constraints are satisfied. This priority ordering – hard geometric/structural feasibility strictly before CoG optimality – was a deliberate design decision (see Section ??) and is enforced identically, in principle, across DE, CMA-ES, and PPO, though each solver imple-

ments the weighting and reward shaping slightly differently, as detailed in Chapter ??.

Chapter 4

System Design and Methodology

4.1 Project Timeline and Overall Workflow

The project progressed through six well-defined stages, illustrated in Figure ?? and summarised in Table ??.

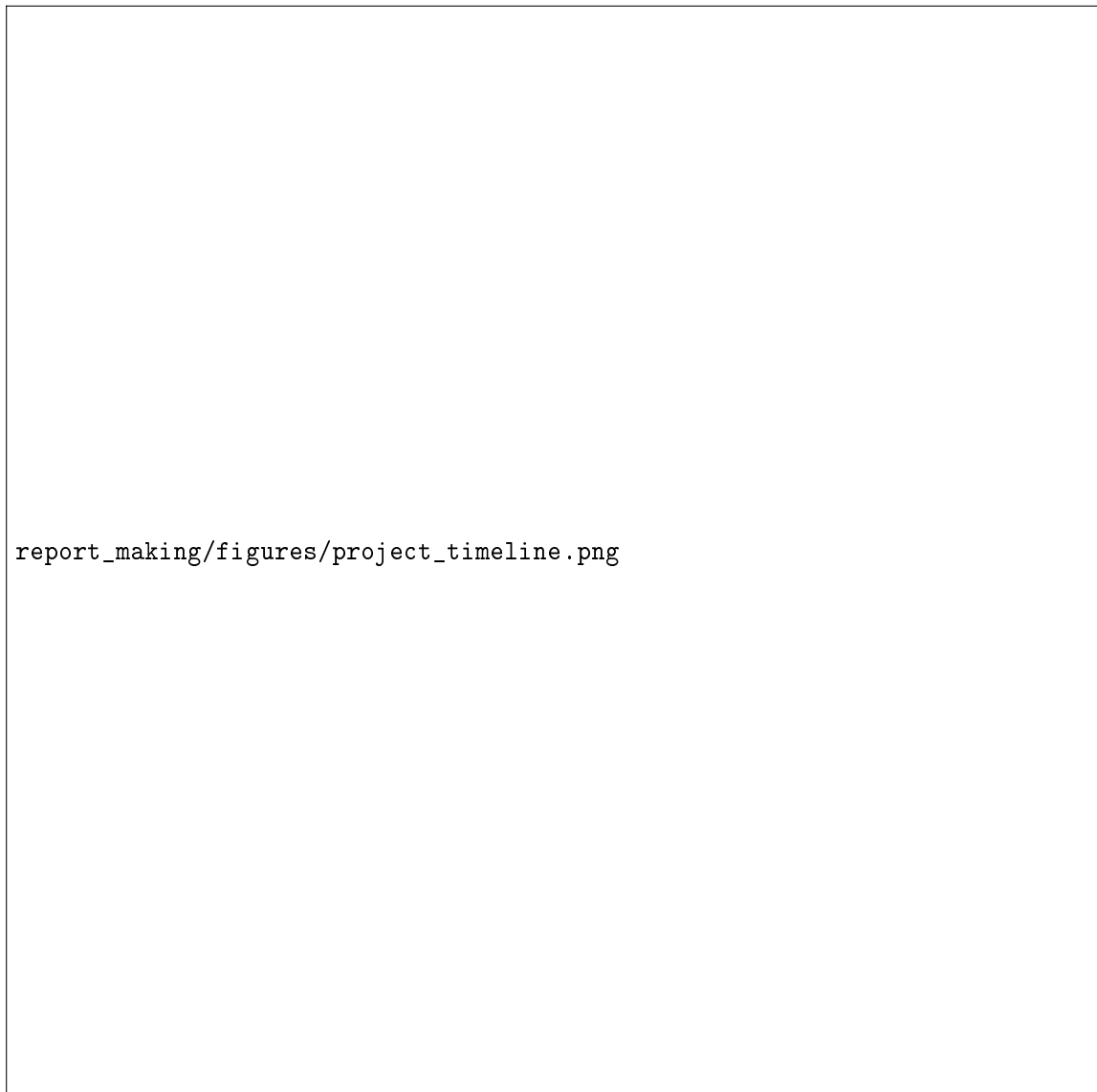


Figure 4.1: Project timeline and workflow, from the guide-supplied DE prototype through to the final multi-algorithm comparative study.

Table 4.1: Stage-by-stage project timeline

Stage	Description
Phase 1	Guide-supplied single-sided DE prototype, iteratively extended with border, clearance, and insert-hole constraints; JSON-driven configuration introduced.
DE pipeline	Full production pipeline (<code>de.py</code>) supporting circular components and dual-sided (front/back) placement, with visualization.
CMA-ES pipeline	Second solver (<code>cma-es-local.py</code>) targeting faster convergence than DE on the same constraint model.
PPO pipeline	Reinforcement-learning solver trained to amortize placement cost across many boards via a trained policy.
Comparative study	Head-to-head benchmarking of DE, CMA-ES, and PPO across computational cost, convergence, scalability, and other axes.
New layout case study	Application of the completed, validated pipeline to a new panel specification (<code>layout2.json</code>) supplied by the project guide.

4.2 Previous Work: Guided DE Prototype

The project began from a Jupyter notebook, `CG_Optimization.ipynb`, whose first cell was supplied directly by the project guide. That cell implements a simple DE-based solver that reads panel dimensions and a small number of components interactively from the console (panel width/height, and per-component name, length, width, and mass) and arranges the components so that the assembly CoG is at the origin, without any collision, border, or insert-hole modelling.

Building on this starting point, three further cells were developed independently, each incrementally adding one constraint layer while retaining full backward compatibility with the CoG objective of the first cell:

1. **Border implementation:** a configurable border margin (e.g. 20 mm) is subtracted from the usable panel from all sides, and every component footprint is constrained to remain within the inset region.
2. **Clearance space:** a configurable minimum clearance distance is added between components on top of the zero-overlap rule; clearance envelopes are permitted to overlap the border but never another component.
3. **Insert holes:** the 4 mm/6 mm insert-hole proximity rule (Table ??) is layered on top of the previous two constraints.

Because manually re-entering component data interactively for every experiment was prohibitively tedious, this data-entry step was replaced with a JSON configuration file (originally `layout1.json`),

and a second notebook, `CG_Optimization_using_json.ipynb`, was written to parse this file directly and run the full four-constraint DE solver end to end, producing five diverse layouts per configuration “for diversity, so that engineers can choose which works best for them.” This established the JSON-driven, five-layouts-per-run convention that all subsequent solvers (DE production pipeline, CMA-ES, PPO) inherited.

4.3 Differential Evolution Pipeline

The production DE pipeline (`DE/de.py`) generalises the previous-work prototype to circular components (in addition to rectangular ones) and to true dual-sided (front + back) placement with a combined visualization showing both faces. The constraint-evaluation core is compiled with Numba’s `@jit(fastmath=True, cache=True)` decorator (Listing ??) so that the CoG, overlap, border, and insert-distance checks – which must be evaluated once per candidate solution, for every member of the DE population, at every generation – run at native machine-code speed rather than interpreted Python speed.

```

1 CG_TOLERANCE = 1.0 # mm: allowed combined CoG offset from origin
2
3 @jit(fastmath=True, cache=True)
4 def compute_fitness_core(cx, cy, masses, hl, hw, clearance,
5                           req_d_matrix, num_components,
6                           flat_offsets, offset_counts, offset_indices,
7                           is_back_side, clearance_dirs,
8                           panel_width, panel_height, border_spacing):
9
10     total_mass = 0.0
11     cg_x = 0.0
12     cg_y = 0.0
13     abs_x = np.zeros(num_components)
14     for i in range(num_components):
15         m = masses[i]
16         total_mass += m
17         if is_back_side[i]:
18             abs_x[i] = -cx[i] # mirror BACK-side x for shared CoG
19         else:
20             abs_x[i] = cx[i]
21         cg_x += abs_x[i] * m
22         cg_y += cy[i] * m
23     if total_mass == 0.0:
24         return 1e15 # reject degenerate solutions
25     cg_x /= total_mass
26     cg_y /= total_mass
27     # ... border, overlap, and insert-distance penalty accumulation ...

```

Listing 4.1: Numba-accelerated fitness core (excerpt from `DE/de.py`)

The DE search itself is invoked through SciPy’s `differential_evolution` routine using the `best1bin` strategy, deferred (batched) updating, and full CPU parallelism across the population, as shown in Listing ??.

```

1 result = differential_evolution(

```

```

2     evaluate, bounds,
3     args=(masses, lengths, widths, clearance, req_d_matrix,
4           flat_offsets, offset_counts, offset_indices,
5           is_back_side, clearance_dirs,
6           panel_width, panel_height, border_spacing),
7     maxiter=600, popsize=25, tol=1e-6,
8     strategy='best1bin', seed=seed,
9     workers=-1, updating='deferred'
10 )
11 if result.fun > 1e-3 or np.any(np.isinf(result.x)) or np.any(np.isnan(result.x)
12 ):
13     continue # strictly reject non-compliant candidates

```

Listing 4.2: DE solver invocation (excerpt from `DE/de.py`)

4.3.1 DE Hyperparameters

Table ?? lists the DE hyperparameters used in production runs, together with the rationale for each, as documented during the project’s performance-bottleneck analysis.

Table 4.2: Differential Evolution hyperparameters and rationale

Parameter	Value	Rationale
Iterations (<code>maxiter</code>)	600	Each iteration evaluates every individual in the population, which can be computationally intensive if the evaluation function is complex.
Population size (<code>popsize</code>)	25	Number of candidate solutions maintained during optimization; balances exploration and exploitation of the solution space.
Workers	-1 (all cores)	Uses all available CPU cores to parallelise evaluation, at the cost of some parallel-processing overhead when the evaluation function does not scale perfectly across workers.
Updating strategy	<code>deferred</code>	All individuals are evaluated before any updates occur within a generation; more conservative but more stable than an <code>immediate</code> update strategy.
Strategy	<code>best1bin</code>	Standard binomial-crossover, best-vector-relative mutation strategy.
Acceptance tolerance	10^{-3} (post-hoc filter)	Candidate solutions with residual penalty $> 10^{-3}$, or containing <code>inf</code> / <code>nan</code> values, are rejected outright rather than accepted as “approximately valid.”

4.4 CMA-ES Pipeline

The CMA-ES pipeline (`CMA-ES/cma-es-local.py`) was introduced after the DE pipeline to investigate whether a covariance-adaptive search could reach fully valid layouts faster than DE, particularly by escaping the local “geometric deadlocks” that plain DE occasionally became stuck in during development. The CMA-ES pipeline reuses the same JSON configuration schema, the same Numba-accelerated constraint core, and the same convention of generating five valid layouts per run, differing from the DE pipeline only in the search algorithm itself. Table ?? lists the key CMA-ES configuration.

Table 4.3: CMA-ES hyperparameters

Parameter	Value	Notes
Initial step size σ_0	15.0 mm	Initial standard deviation of the search distribution.
Population size	32	Candidate solutions generated per generation.
Restarts	multiple, randomised	Multiple random restarts are run to produce up to five distinct valid layouts.

As documented in Section ??, CMA-ES’s adaptive covariance-matrix scaling was specifically found to resolve local-minima traps that plain DE could not escape, where components sat close to a valid configuration but could not move without transiently increasing overlap penalty.

4.5 PPO Reinforcement Learning Pipeline

The PPO pipeline is the most substantial engineering effort of the project, replacing the per-board evolutionary search of DE/CMA-ES with a trained neural-network policy that can generate new, valid layouts for previously unseen problem instances via a single forward pass.

4.5.1 Motivation

The original single-sided reference solution used CMA-ES to score any candidate layout using the shared penalty formulation of Equation ??, then ran multiple random restarts to obtain up to five distinct valid layouts. The goal of the PPO rewrite was to replace this per-board evolutionary search entirely with an RL agent trained once, offline, whose trained weights can then be reused to generate arbitrarily many new layouts near-instantly, in contrast to DE/CMA-ES, which must always re-run their full search loop from scratch.

4.5.2 MDP Formulation and Environment

The placement task is formulated as an episodic *iterative-refinement* MDP, implemented as a custom `gymnasium.Env` subclass (`PCBPlacementEnv`). Three candidate episode/action structures were considered during design (Table ??); iterative refinement was selected because it allows the

policy to make large corrective moves early in an episode and small, precise moves later, closely mirroring how a human designer nudges components into place.

Table 4.4: PPO design decisions

Decision point	Options considered	Chosen
Episode/action structure	(a) one-shot full-layout output; (b) sequential one-component-at-a-time placement; (c) iterative refinement	(c) Iterative refinement
Hybridization	(a) pure PPO; (b) PPO + CMA-ES polish per episode; (c) PPO global search + CMA-ES fallback	(a) Pure PPO only
Reward shaping	(a) sparse; (b) dense	(b) Dense

At the start of every episode, all components are initialised to random valid-bounds positions. At every timestep, the policy outputs a small $(\Delta x, \Delta y)$ nudge for *every* component simultaneously; the magnitude of this nudge is annealed within the episode from `MAX_STEP_MM` = 12.0 mm at the start down to `MIN_STEP_MM` = 0.25 mm towards the end, permitting large corrective movements early and fine-grained positioning later without overshoot. Episodes are truncated after `MAX_STEPS` = 250 timesteps if a fully valid layout has not been found, and terminate immediately, with a large terminal bonus, the instant a layout satisfies every hard constraint.

4.5.3 Reward Function Design

The composite, log-compressed penalty score used to derive the reward signal is

$$\text{composite} = 3.0 \cdot \log(1 + P_{\text{border}}) + 3.0 \cdot \log(1 + P_{\text{overlap}}) + 10.0 \cdot \log(1 + P_{\text{insert}}) + 0.5 \cdot \log(1 + P_{\text{cg}}), \quad (4.1)$$

$$r_t = (\text{composite}_{t-1} - \text{composite}_t) - \text{STEP_COST} + \text{SUCCESS_BONUS} \text{ (on success step only)}, \quad (4.2)$$

with `STEP_COST` = 0.01 applied at every step to encourage efficient solutions, and `SUCCESS_BONUS` = 50 awarded once, on the exact step a layout first becomes fully valid. The $\log(1 + P)$ compression is essential because raw overlap/insert penalties can reach 10^9 – 10^{10} in early, highly invalid episodes, which would otherwise destabilise PPO’s value-function learning. The relative weighting was retuned mid-project: border and overlap weights were reduced from 5.0 to 3.0 once those constraints were reliably being solved, while the insert-hole weight was raised from 5.0 to 10.0 after it was identified as the hardest constraint to satisfy exactly, and the CG weight was kept lowest (0.5) to reflect its status as the soft, priority-2 objective. This weighting scheme also had to specifically defeat a reward-hacking failure mode in which the agent could trivially satisfy the CoG penalty by collapsing all components to the origin while ignoring overlap – solved by ensuring hard physical-violation penalties always dominate the soft CoG term, as discussed

further in Chapter ??.

4.5.4 Network Architecture and Hyperparameters

The PPO policy uses Stable-Baselines3’s `MlpPolicy` with the architecture and hyperparameters summarised in Table ?? and Table ??.

Table 4.5: PPO Actor–Critic network architecture

Component	Specification
Actor (policy) network	Fully connected MLP, 2 hidden layers of 256 neurons each; maps observed state directly to placement actions.
Critic (value) network	Matching MLP architecture (2 hidden layers of 256 neurons); estimates the state-value function.
Activation function	Hyperbolic tangent ($\tanh$), range $[-1, 1]$, zero-centred, for smooth gradient flow across hidden layers.
Optimizer	Adam, initial learning rate $\alpha = 3 \times 10^{-4}$.

Table 4.6: PPO training hyperparameters

Hyperparameter	Value	Purpose
Clipping threshold ϵ	0.2	Restricts each policy update to a relative range of $0.8\text{--}1.2\times$ the previous policy, preventing destructive updates.
Rollout buffer size	1,024 steps/env	Interaction steps collected per parallel environment before an update.
Mini-batch size	256	Size of mini-batches drawn from the rollout buffer during optimization.
Epochs per update	10	Number of optimization passes over each collected rollout buffer.
Discount factor γ	0.99	Prioritises long-term layout optimality over immediate reward.
GAE factor λ	0.95	Generalized Advantage Estimation smoothing; balances variance reduction with bias control.
Entropy coefficient c_1	0.005	Encourages exploration and prevents premature convergence.
Value-loss weight c_2	0.5	Weight of MSE value-function loss relative to the policy objective.
Max gradient norm	0.5	Gradient clipping for stable back-propagation.
Total training budget	1,000,000+ steps	Continuous training over environment interactions (extended to 6M steps in the final checkpoint).
Parallel environments	8 (<code>SubprocVecEnv</code>)	Training parallelism; observations/rewards normalised via <code>VecNormalize</code> .
Episode horizon	250 steps	<code>MAX_STEPS</code> before forced truncation.
Step cost / success bonus	0.01 / 50	See reward formulation, Section ??.

4.5.5 Training Infrastructure and Execution

Training used `SubprocVecEnv` with 8 parallel environments plus `VecNormalize` for observation/reward normalization; inference used a `DummyVecEnv` with 8 environments re-using the same saved `VecNormalize` statistics with `training=False`. All geometric constraint evaluation remained on CPU via Numba JIT, while the PPO neural network itself ran on Apple Silicon’s Metal Performance Shaders (MPS) backend. TensorBoard logging (`tb_logs/`) was used throughout to monitor training metrics, and a custom `BestLayoutCallback`, inheriting from Stable-Baselines3’s `BaseCallback`, continuously tracked the best-performing layout found during training.

4.5.6 Multi-Layout Inference Pipeline

Once trained, generating layouts is a pure inference operation: the trained model and its `VecNormalize` statistics are loaded, many deterministic rollouts of the policy are executed from independent random resets, each candidate layout is independently re-verified against the exact (non-learned) constraint functions, near-duplicate layouts are removed via a mean per-component displacement threshold (`DEDUP_DIST_MM`), and up to `MAX_LAYOUTS = 5` distinct layouts are kept, ranked by CoG penalty. This entire pipeline is also exposed as a standalone command-line script, `gen_from_inference.py`, configurable via `-episodes`, `-parallel`, `-max_layouts`, `-dedup_dist`, and `-seed` flags, so that new layouts can be regenerated without opening the training notebook.

4.5.7 Training Dataset

The `PPO/datasets/` directory contains 1,049 valid layouts stored in `.npz` format, generated during earlier stages of the project. These represent previously discovered valid configurations of the underlying assembly and are retained alongside the PPO pipeline as a reference/warm-start resource, although the final PPO policy reported in this project was trained purely through environment interaction (self-play against the reward function) rather than supervised imitation of this dataset.

4.6 Comparative Study Methodology

Once all three solvers were independently validated, a dedicated comparative study (`comparission/run_comparative_study.py`) was implemented to benchmark them head-to-head under identical problem instances. The study measures, per algorithm: wall-clock time to solve a single board; fitness-penalty trajectory over normalised search progress; behaviour as the number of components scales from 4 to 20+; statistical significance of pairwise performance differences over repeated runs; the spatial distribution of chosen component placements; the marginal cost of generating additional boards once initial setup/training cost has been paid; and the decomposition of any residual penalty into its border/overlap/insert/CoG components. The convergence-trajectory analysis specifically was computed and finalised in a companion notebook, `comparission/convergence_trajectory.ipynb`, to obtain a clean, variance-shading-free comparison of how rapidly each algorithm reduces its penalty score. Full results of this study are presented in Section ??.

Chapter 5

Implementation

5.1 Tools, Libraries, and Environment

Table ?? summarises the software stack used across the project.

Table 5.1: Software stack

Category	Tools / Libraries
Numerical optimization	SciPy (<code>differential_evolution</code>), <code>cma</code> (CMA-ES reference implementation)
JIT acceleration	Numba (<code>@jit</code> , <code>prange</code>) for constraint/fitness evaluation
Reinforcement learning	Gymnasium (custom <code>Env</code>), Stable-Baselines3 (PPO, <code>MlpPolicy</code>), PyTorch
Data & configuration	NumPy, JSON (board/component schema), <code>.npz</code> (layout datasets)
Visualization	Matplotlib (Agg backend), custom rectangle/circle patch rendering
Experiment tracking	TensorBoard (<code>tb_logs/</code>)
Development environment	Jupyter Notebook (offline / no-internet-access development environment; see Chapter ??)

5.2 Development Environment Constraints

A defining engineering constraint of this project, discussed further in Chapter ??, was that the primary development machine had no internet access and could not install external software or additional Python packages beyond what was already available in the pre-installed Jupyter environment. All algorithms, custom penalty logic, and visualization code were consequently authored, tested, and iterated on a separate internet-connected machine before being transferred into the restricted environment.

5.3 Fitness Evaluation Core

The single most performance-critical component of the entire pipeline is the pairwise constraint-evaluation core, since it must be invoked once per candidate solution, for every member of a population (25 for DE, 32 for CMA-ES), at every one of hundreds of generations, and again once

per environment step across 8 parallel PPO training environments. All three solvers therefore share the same Numba-compiled evaluation strategy: geometry is expressed as flat NumPy arrays (not Python objects) so that Numba’s `nopython` mode can compile the full loop nest to native code, and the CoG, border, overlap, and insert-distance sub-penalties are all computed within a single fused pass over the component list to minimise redundant memory traffic. This design choice was central to keeping wall-clock optimization time practical: without JIT compilation, evaluating thousands of candidate component arrangements per second for overlap, border, and insert-hole spacing in interpreted Python would have made both DE and CMA-ES impractically slow, and would have made 8-way parallel PPO training infeasible on the development hardware.

5.4 Visualization Pipeline

Every solver shares a common Matplotlib-based visualization pipeline (using the headless `Agg` backend so that figures can be rendered without a GUI, suitable for batch/offline execution) that produces, per layout:

- a **front/back projection view**, in which rectangular components are drawn as rectangle patches and circular components as circle patches, coloured/labelled by name, with front-face and back-face components shown together for a full assembly picture;
- a **proximity / distance map**, visualising the pin-to-pin distances between insert holes to allow visual sanity-checking of the insert-proximity constraint;
- a **tabular data export** (CSV) listing, per component: name, face, mass, dimensions, and its individual CoG (for PPO only).

For the PPO pipeline specifically, the proximity map was refined during development into a true “see-through” rendering: back-layer components are drawn first with reduced opacity ($\alpha = 0.30$), hatched fill, and lower z-order (ghosted), while front-layer components are drawn second at full opacity ($\alpha = 0.85$) and higher z-order, with insert-hole pin markers colour-coded by layer (red = front, orange = back) so that overlapping pins on opposite faces remain visually distinguishable. This is documented further, with worked examples, in Chapter ??.

5.5 Reproducibility

Every solver run is seeded (DE via SciPy’s `seed` argument, CMA-ES via its internal RNG seed, and PPO’s inference rollouts via a `-seed` flag defaulting to 42) so that reported results and figures can, in principle, be regenerated exactly. All generated layouts are additionally independently re-verified against the ground-truth (non-learned) constraint functions before being accepted, which is particularly important for the PPO pipeline since the trained policy’s outputs are otherwise unconstrained neural-network predictions.

Chapter 6

Results and Discussion

6.1 Previous-Work Baseline

Figure ?? presents representative console interaction and output from the original guided DE prototype (Section ??), used as the functional baseline against which every subsequent constraint layer and solver was validated. The prototype accepts, for a 5-component test assembly on a 200×200 mm panel, the panel dimensions and per-component name/length/width/mass, and arranges the components to null the CoG without any collision, border, or insert-hole modelling.

Table 6.1: Baseline 5-component test assembly used to validate the previous-work prototype

Element	Length (mm)	Width (mm)	Mass (kg)
PCU	30	40	1.5
OBC	50	50	2.3
SPS	20	20	0.8
RPU	60	30	0.7
RS	20	20	1.9

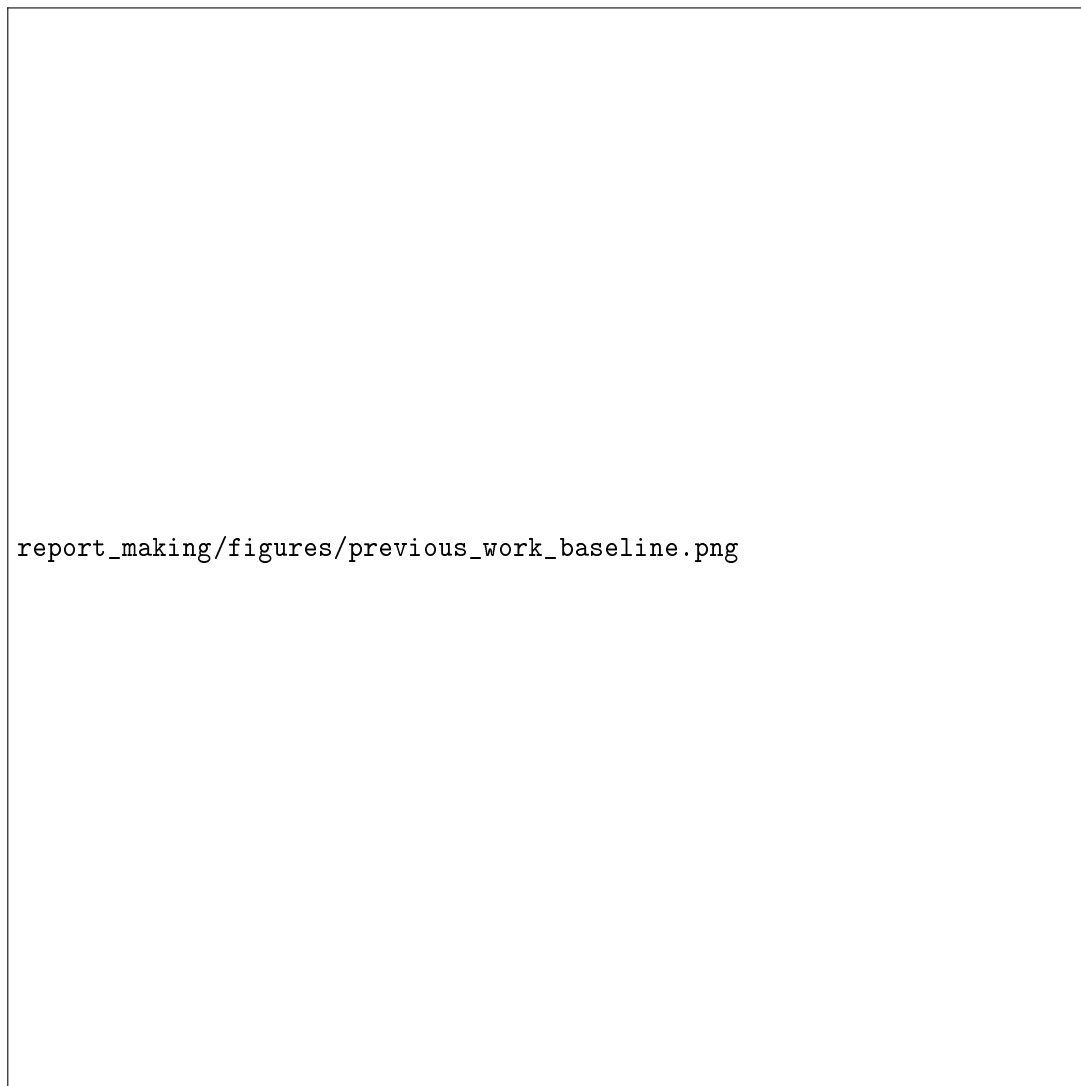
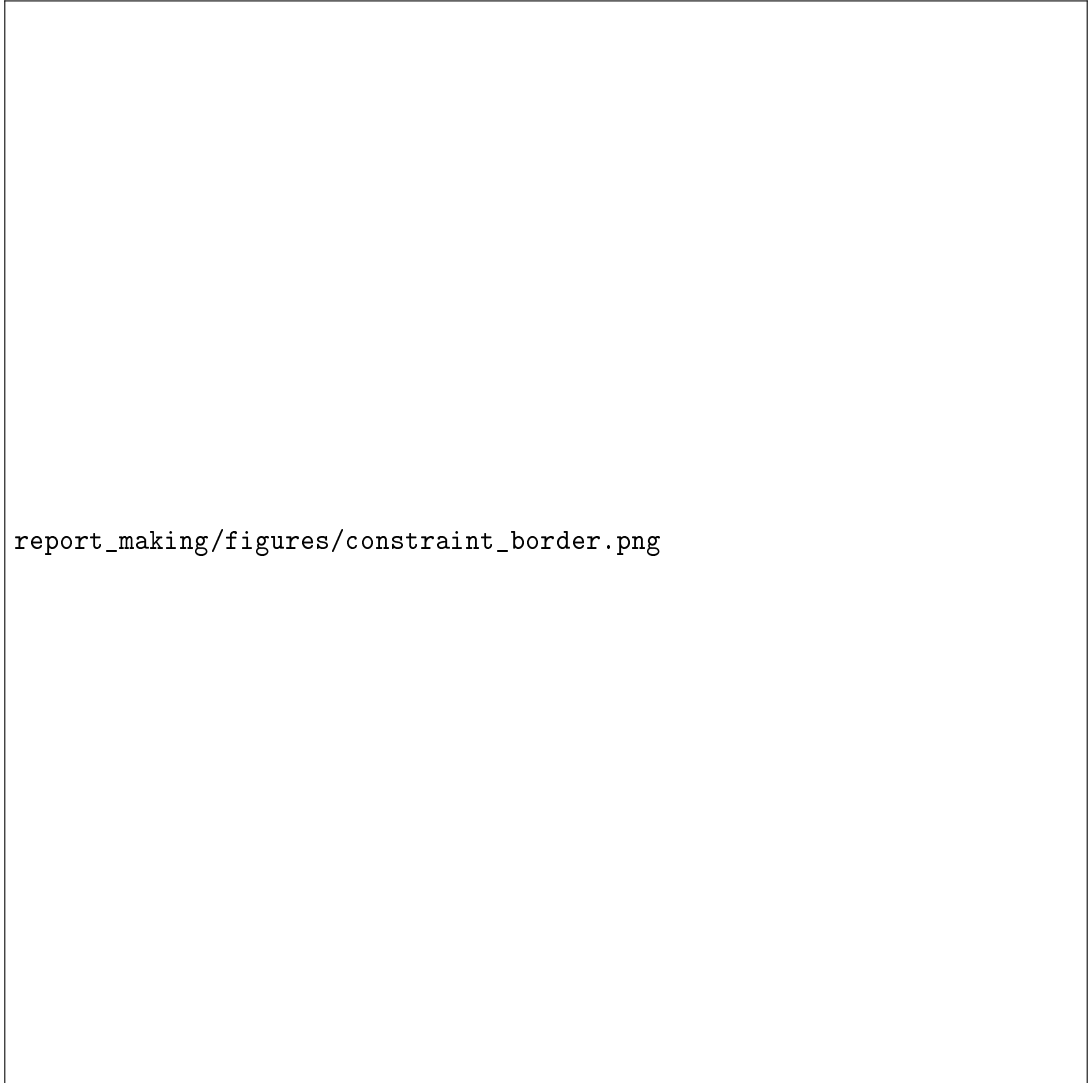


Figure 6.1: Baseline CoG-only placement output from the original guided DE prototype.

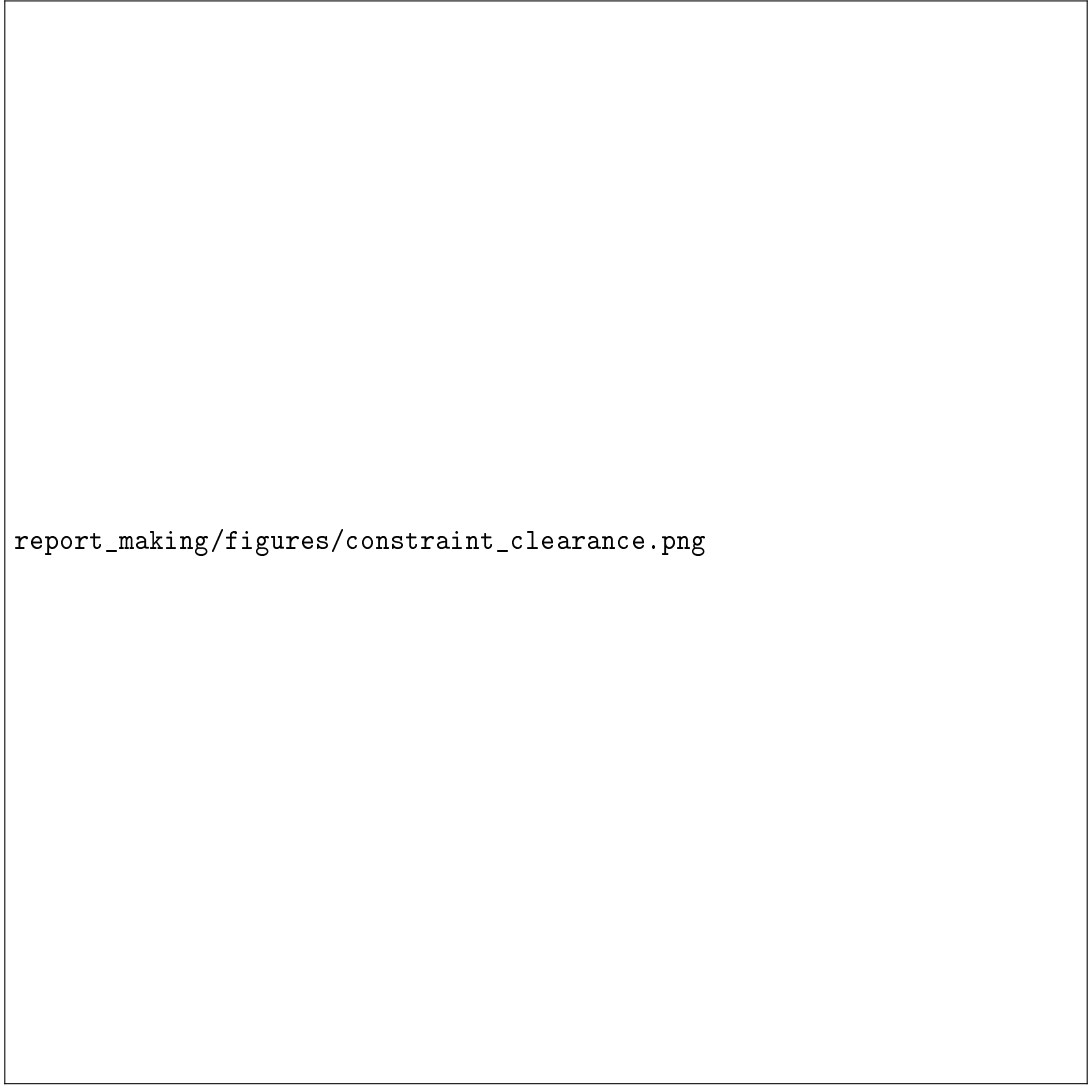
6.1.1 Constraint Verification: Border, Clearance, and Insert Holes

Figures corresponding to Slides 8, 12, and 14 of the project presentation visually confirm, in turn, that the border margin (20 mm on the baseline 200×200 mm panel), the same-face clearance/Connector-Face rule, and the 4/6 mm insert-hole proximity rule are each correctly enforced on the baseline assembly before progressing to the full production pipeline.



`report_making/figures/constraint_border.png`

Figure 6.2: Verification of border-containment constraint on the baseline assembly.



report_making/figures/constraint_clearance.png

Figure 6.3: Verification of clearance and connector-face constraint on the baseline assembly.

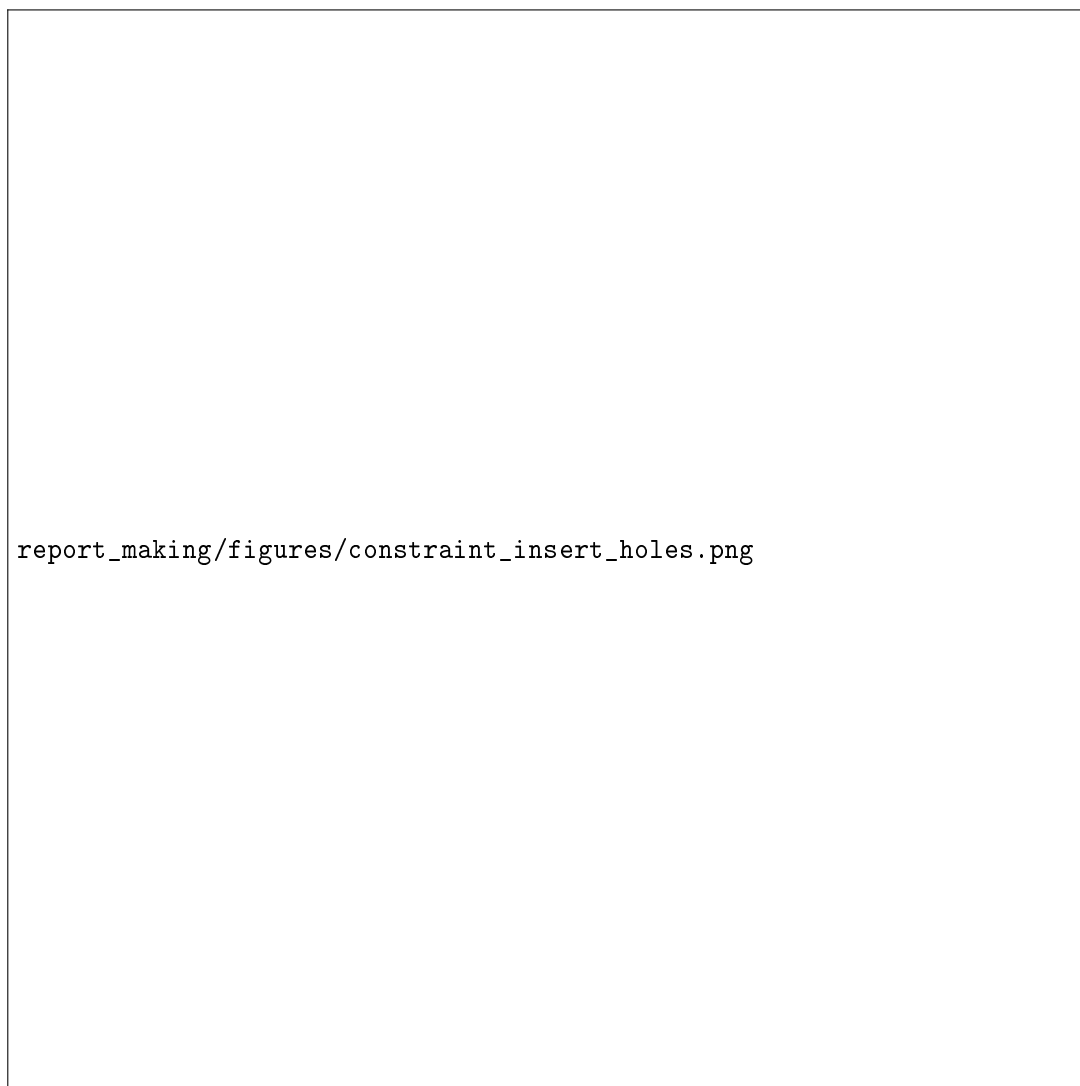


Figure 6.4: Verification of insert-hole minimum-separation constraint on the baseline assembly.

Old and New Layout Specifications

The production results are evaluated on two distinct configurations. **Layout1** is the original configuration used during the initial development and testing of the placement pipeline, while **Layout2** is the newer configuration worked on subsequently. The tables below reproduce the specifications supplied in the corresponding XLSX files so that the results in the following sections can be interpreted against the correct problem instance.

Table 6.2: Old layout (Layout1) board and component specification

Name	L	B	Mass	Insert	Ins. Qty.	Obj. Qty.	CF	Shape/Dia.
Board	960	582.4	–	–	–	–	–	–
PDS	100	112	1.5	4	4	1	[1,3]	Rectangle
CCRR	–	–	0.15	6	6	2	–	Circle, $\varnothing 45$
LD	–	–	0.04	6	3	6	–	Circle, $\varnothing 32$
RS	110	90.5	1.5	4	4	1	[1,3]	Rectangle
BM1	100	100	1.5	6	4	1	[1]	Rectangle
PB	69	16	0.1	4	2	4	[1]	Rectangle
RPU	260	253	3	4	6	1	[2,4]	Rectangle
MES	128	117	0.125	6	4	1	[3,4]	Rectangle
MT20	275	25	0.4	4	4	1	[1]	Rectangle

L = Length (mm), B = Breadth (mm), Mass in kg. The XLSX identifies PDS, CCRR, LD and RS under Front, and BM1, PB, RPU, MES and MT20 under Back.

Table 6.3: New layout (Layout2) board and component specification

Name	L	B	Mass	Insert	Ins. Qty.	Obj. Qty.	FC
Board	3285	1458	–	–	–	–	–
Bat10	557	268	37.5	6	24	1	1
Bat20	557	268	37.5	6	24	1	1
IRU	366	268	13	6	6	1	1
OBC10	362	252	16	6	24	1	1
OBC20	362	252	16	6	24	1	1
ICP	286	337	14.9	4	20	1	–
SPDU	376	304	9	4	10	1	0,1,2,3
FDM10	224	51	3	4	6	1	0,1,2,3
FDM20	224	51	3	4	6	1	0,1,2,3
IAPM1	250	250	14	6	6	1	0,1,2,3
IAPM2	250	250	14	6	6	1	0,1,2,3
IAPR1	250	250	14	6	6	1	0,1,2,3
IAPR2	250	250	14	6	6	1	0,1,2,3
BMU51	106	97	1.5	4	4	1	1
BMU52	106	97	1.5	4	4	1	1
BMU53	106	97	1.5	4	4	1	1
BMU54	106	97	1.5	4	4	1	1
SSPA10	120	195	3.5	4	6	1	0,2
SSPA20	120	195	3.5	4	6	1	0,2
SSPA30	120	195	3.5	4	6	1	0,2
SSPA40	120	195	3.5	4	6	1	0,2
KABANDTRT10	180	220	4	4	4	1	1,4
KABANDTRT20	180	220	4	4	4	1	1,4
SAGS	120	90	2	4	4	1	0,2
TRM10	182	75	1.8	4	4	1	0,2
TRM20	182	75	1.8	4	4	1	0,2
TRM30	182	75	1.8	4	4	1	0,2
TRM40	182	75	1.8	4	4	1	0,2
TRM50	182	75	1.8	4	4	1	0,2
TRM60	182	75	1.8	4	4	1	0,2

L = Length (mm), B = Breadth (mm), Mass in kg. The FC values are reproduced exactly as supplied in the Layout2 XLSX. The source sheet does not specify FRONT/BACK face labels.

6.2 Differential Evolution Results

The Differential Evolution (DE) results are presented for both configurations so that the effect of moving from the original working problem to the recently developed Layout2 problem can be seen directly. **Figure ??** is the representative output from the old Layout1 configuration, whereas **Figure ??** is the corresponding output from the new Layout2 configuration. In each case, the main layout view is paired with its distance-mapping output; therefore, the two figures should be interpreted as two views of the same solver result rather than as independent experiments.

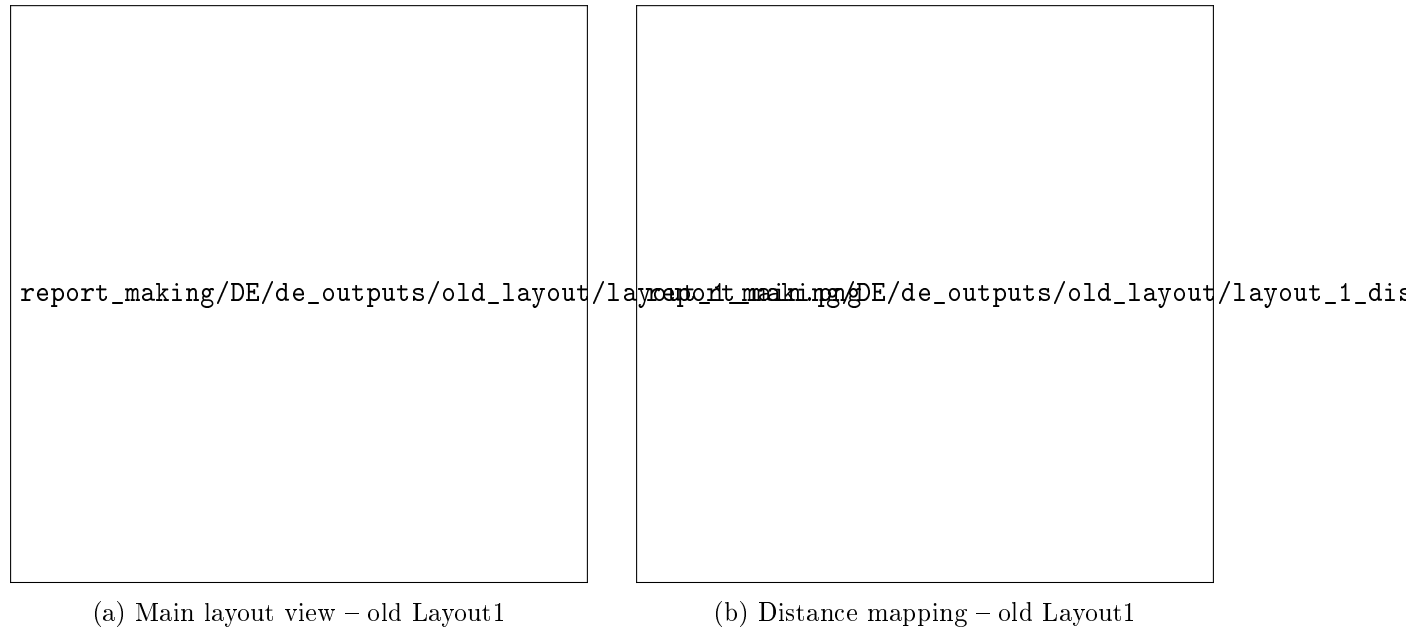


Figure 6.5: Differential Evolution output for the old layout (Layout1): main placement and corresponding insert-distance mapping.

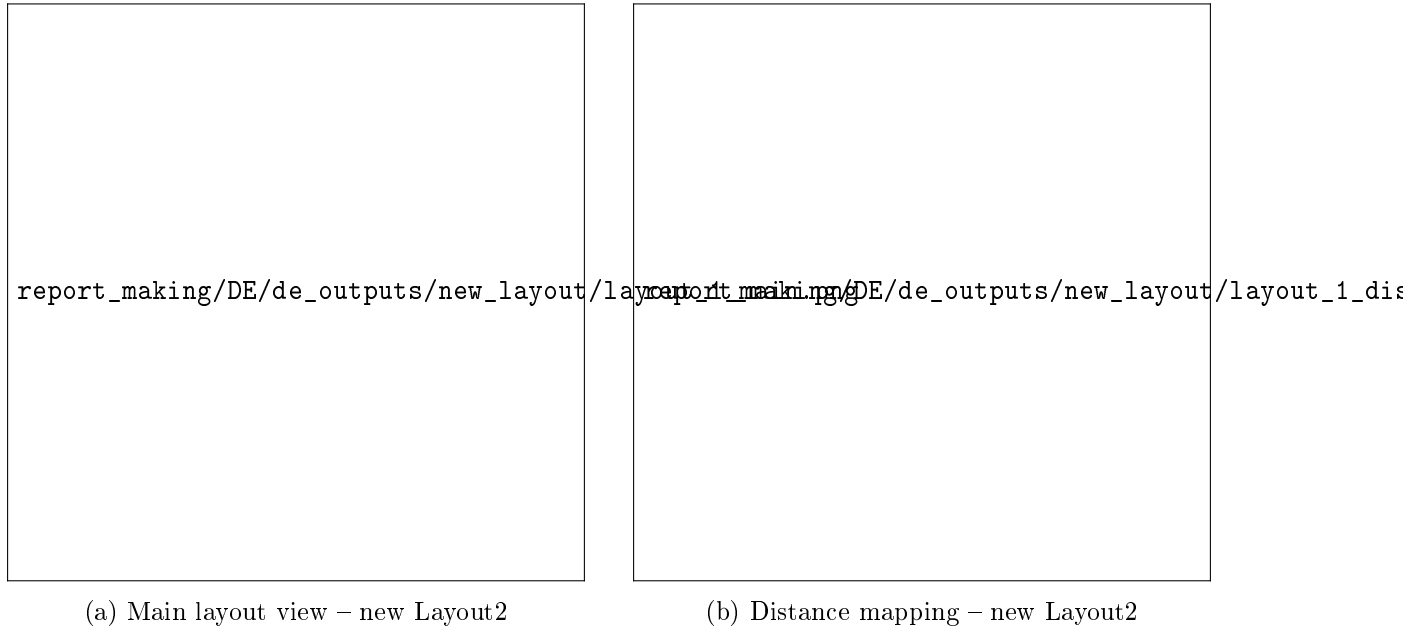


Figure 6.6: Differential Evolution output for the new layout (Layout2): main placement and corresponding insert-distance mapping.

6.2.1 DE Performance Bottleneck Analysis

The DE configuration (600 iterations $\times$ 25 population, all CPU cores, deferred updating) reflects a deliberate trade-off: larger population sizes and iteration counts improve the probability of finding a fully compliant solution but increase wall-clock cost roughly linearly, while parallel-processing overhead across CPU workers can partially offset the benefit of using all available cores when the per-candidate evaluation function does not scale perfectly. The old and new layout results also represent different problem scales: Layout1 is the original, relatively compact test configuration, whereas Layout2 contains a substantially larger panel and a much larger component set. Consequently, the two results should be compared primarily as demonstrations of the pipeline operating on the two configurations rather than as a controlled timing experiment.

6.3 CMA-ES Results

CMA-ES was evaluated using the same two layout configurations and the same physical constraint model. **Figure ??** shows a representative result for the old Layout1 configuration, while **Figure ??** shows the corresponding result for the new Layout2 configuration. As with DE, each figure contains both the main placement view and the associated distance map so that component placement and insert-distance compliance can be inspected together.

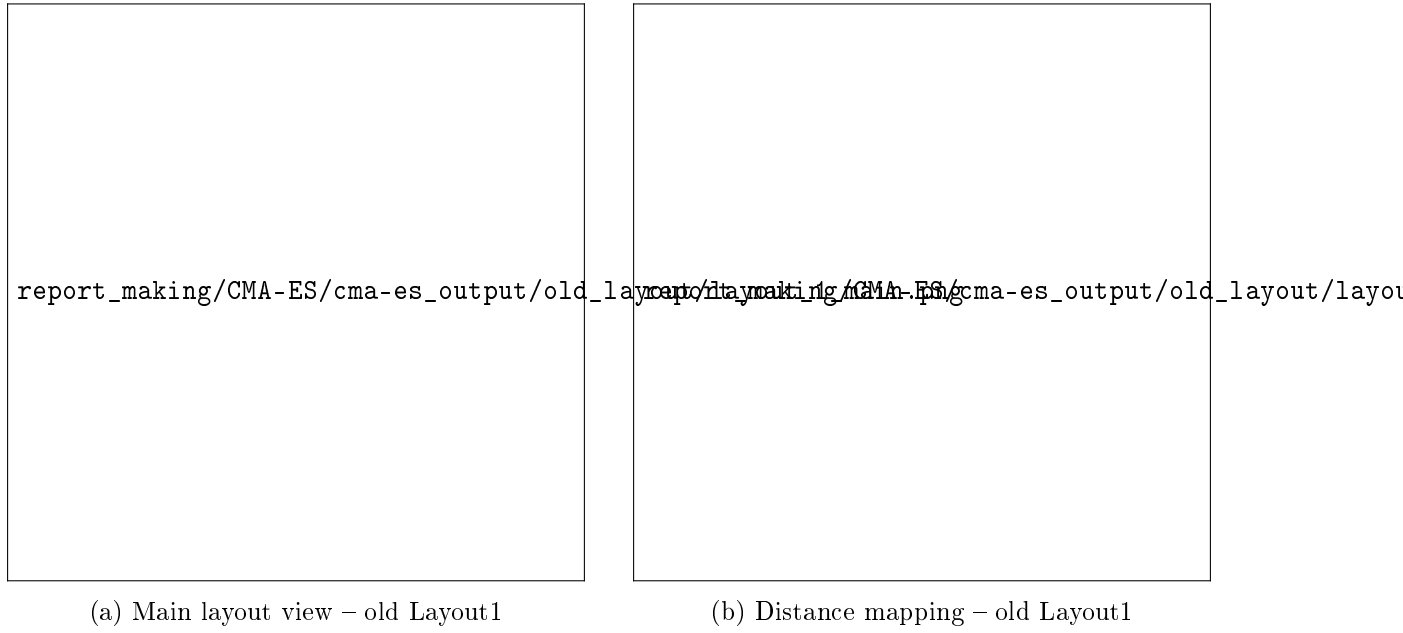


Figure 6.7: CMA-ES output for the old layout (Layout1): main placement and corresponding insert-distance mapping.

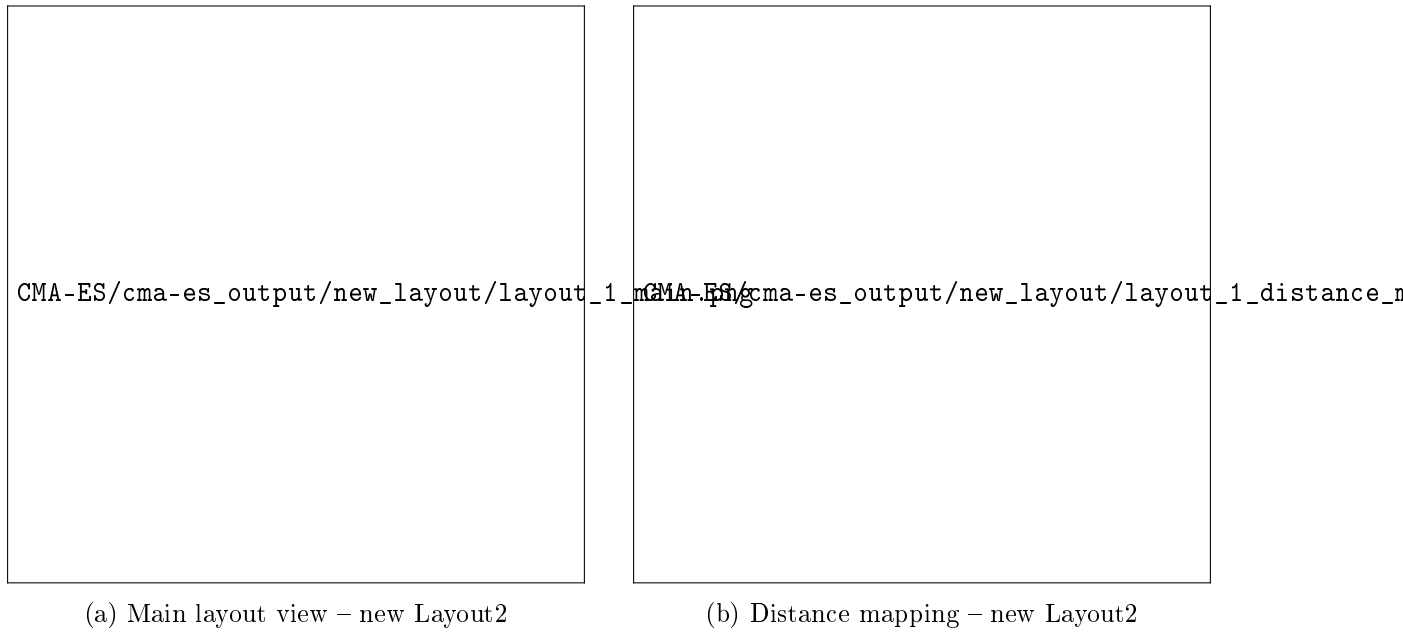


Figure 6.8: CMA-ES output for the new layout (Layout2): main placement and corresponding insert-distance mapping.

Qualitatively, CMA-ES was observed during development to converge to a fully valid layout more reliably than plain DE on configurations where components were densely packed relative to the panel area, consistent with its adaptive covariance-matrix mechanism being specifically effective at escaping the local “geometric deadlocks” documented in Chapter ?? . The Layout2 result demonstrates that the same CMA-ES pipeline can be applied to the newer configuration without changing the underlying optimization formulation.

6.4 PPO Results

The PPO results are likewise presented as an old-layout/new-layout pair. **Figure ??** represents the final inference output for the original Layout1 problem, while **Figure ??** represents the corresponding output for the newer Layout2 problem. Each result combines the main placement visualization with its see-through distance mapping, allowing the physical placement and insert-distance relationships to be inspected together. The PPO result tables previously included in this section have been removed as requested; the visual results are retained because they directly show the layouts produced by the trained policy and their geometric verification.

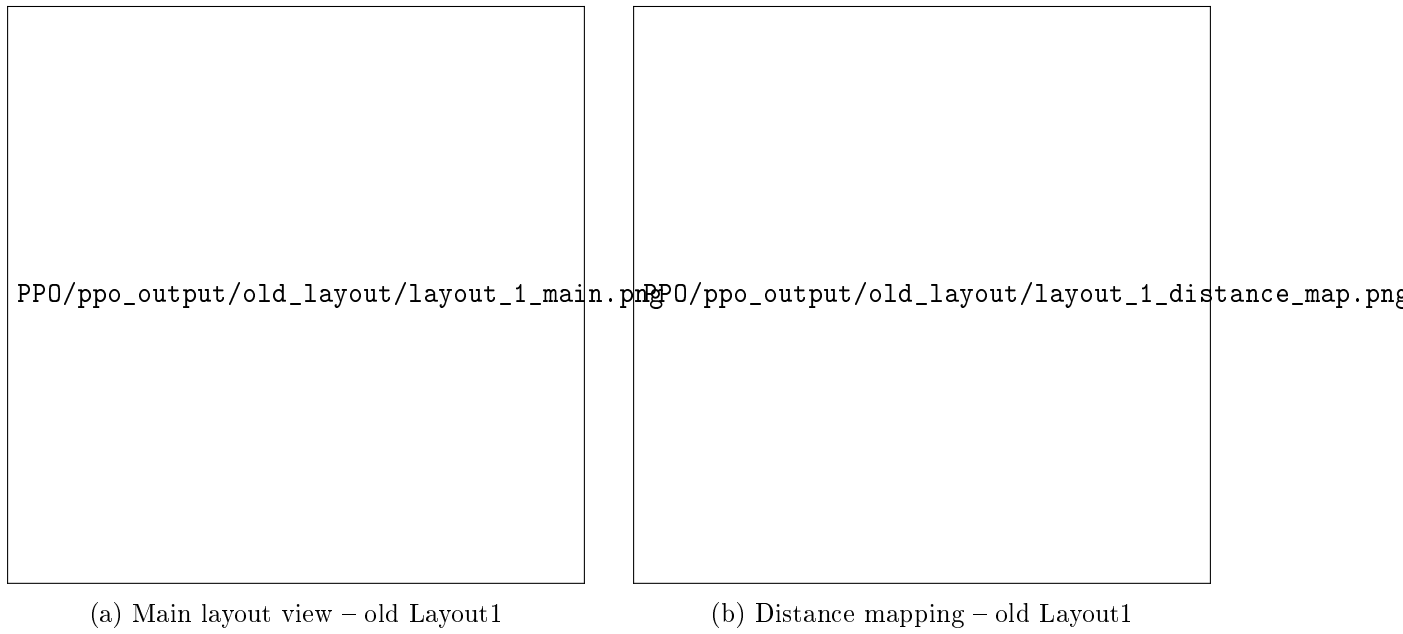


Figure 6.9: PPO output for the old layout (Layout1): main placement and corresponding see-through insert-distance mapping.

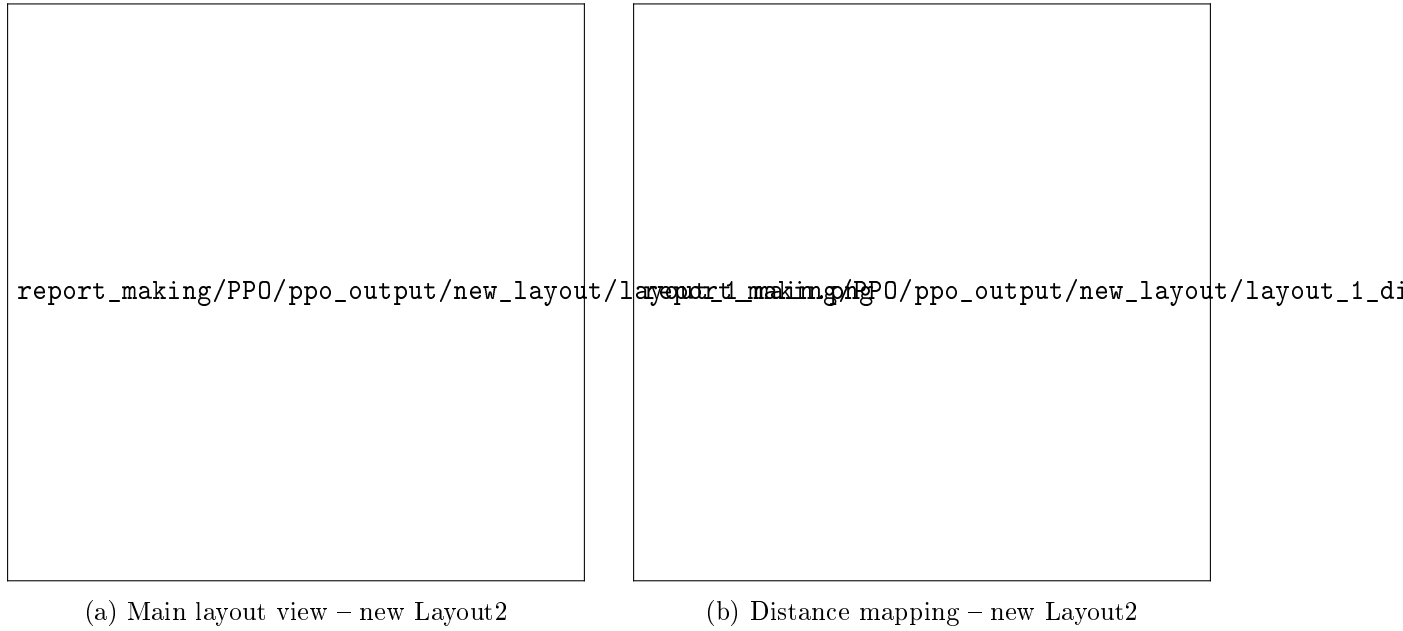


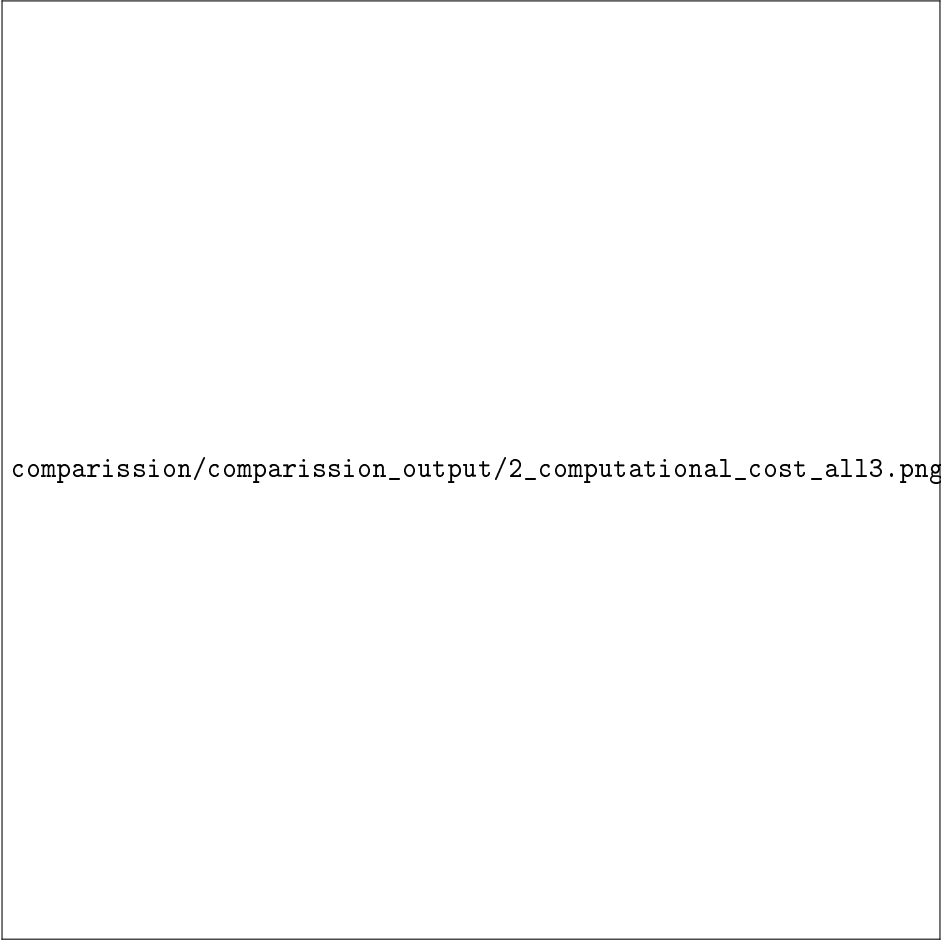
Figure 6.10: PPO output for the new layout (Layout2): main placement and corresponding see-through insert-distance mapping.

6.5 Comparative Performance Analysis

This section presents the eight comparative results generated by the head-to-head study (Section 4.6). Each result is deliberately kept together with its explanation and figure on a single page so that the reader can interpret the plotted evidence without having to turn to a separate page for the associated discussion.

6.5.1 Computational Cost (Wall-Clock Time)

Figure ?? compares the average execution time, in seconds, required by each algorithm to solve a single board layout. This metric evaluates real-time feasibility for a production engineering workflow. PPO requires approximately 0.02 s per inference step once trained, whereas CMA-ES requires approximately 1.5 s and DE requires approximately 2.5 s, since both metaheuristics must perform iterative objective re-evaluation from scratch for every new board.



comparission/comparission_output/2_computational_cost_all3.png

Figure 6.11: Computational cost (wall-clock time) comparison across DE, CMA-ES, and PPO.

6.5.2 Spatial Placement Density Heatmaps

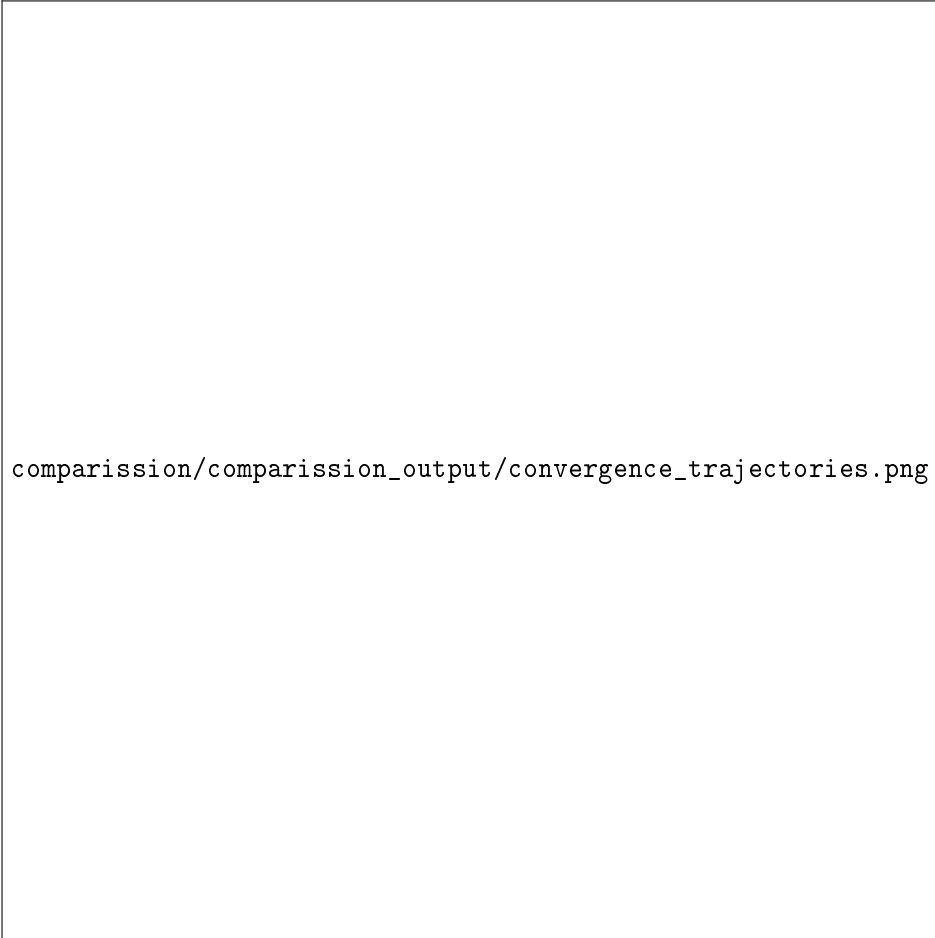
Figure ?? shows side-by-side Kernel Density Estimation (KDE) contour heatmaps of chosen component placements on the panel grid, for all three algorithms. This visualises each algorithm's physical placement bias: DE tends to push components toward the panel's boundary margins, while PPO centres them more safely within the interior of the clearance envelopes.



Figure 6.12: Spatial placement density (KDE) heatmaps for DE, CMA-ES, and PPO.

6.5.3 Convergence Trajectory

Figure ?? plots the mean fitness-penalty score against normalised search progress (0–100%) for all three algorithms, providing a direct, variance-shading-free comparison of how rapidly each algorithm minimises constraint violations and reaches a stable, feasible layout.



comparission/comparission_output/convergence_trajectories.png

Figure 6.13: Convergence trajectory: mean fitness penalty vs. normalised search progress.

6.5.4 Statistical Significance (p-value Heatmap)

Figure ?? presents a pairwise statistical-significance (p-value) heatmap across repeated runs of each algorithm. A p-value below 0.05 indicates that the observed difference between the compared algorithms is statistically significant under the test used in the comparative study.



Figure 6.14: Pairwise statistical significance (p-value heatmap) across DE, CMA-ES, and PPO.

6.5.5 Scalability Analysis

Figure ?? shows execution time as the number of components scales from 4 to 20+. Evolutionary approaches (DE, CMA-ES) scale approximately quadratically to polynomially, in the range $O(N^{1.9})$ to $O(N^{2.2})$, since the pairwise overlap/insert-distance checks and the search space itself both grow with N , whereas PPO inference scales near-linearly, $O(N)$, since it performs a fixed-cost forward pass per component nudge regardless of population-based search overhead.

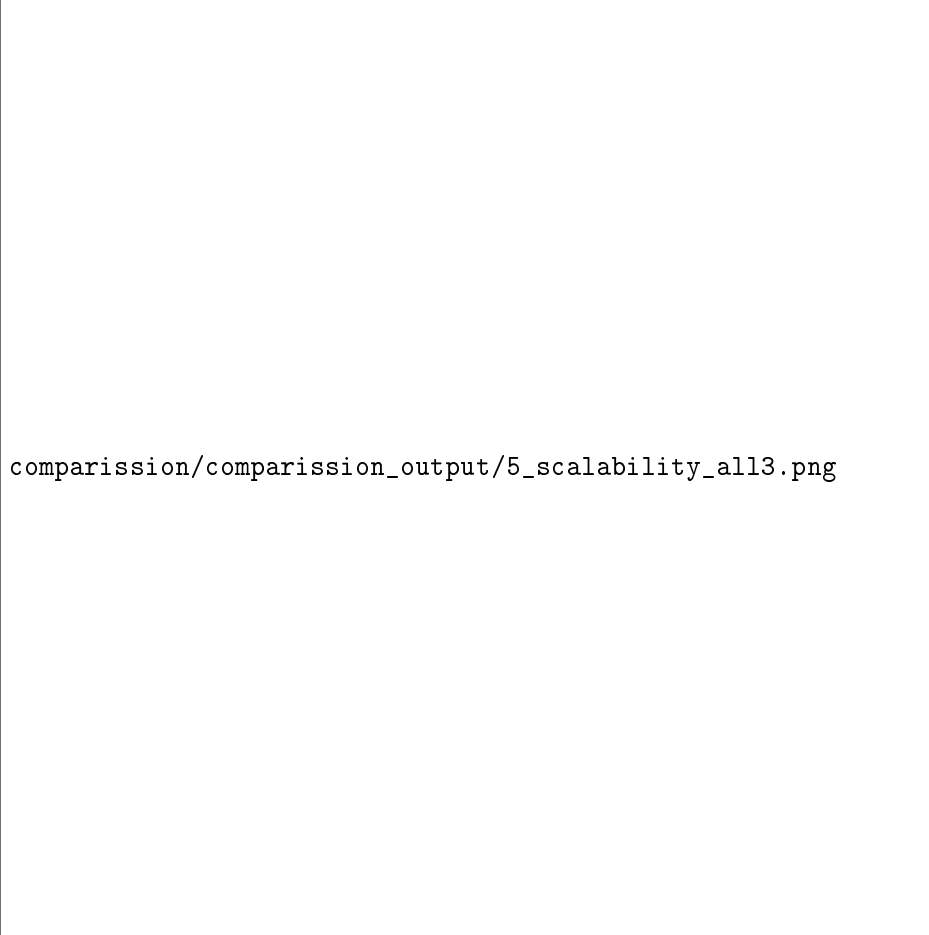


Figure 6.15: Scalability analysis: execution time vs. number of components.

6.5.6 Solution Diversity vs. Quality (Pareto Frontier)

Figure ?? maps spatial component dispersion (diversity) against fitness score (quality) as a scatter plot, identifying the trade-off between finding a single high-quality valid layout and finding a set of varied layouts – directly relevant to the project’s five-layouts-per-run requirement, since engineers benefit from genuinely distinct alternatives rather than five near-identical solutions.

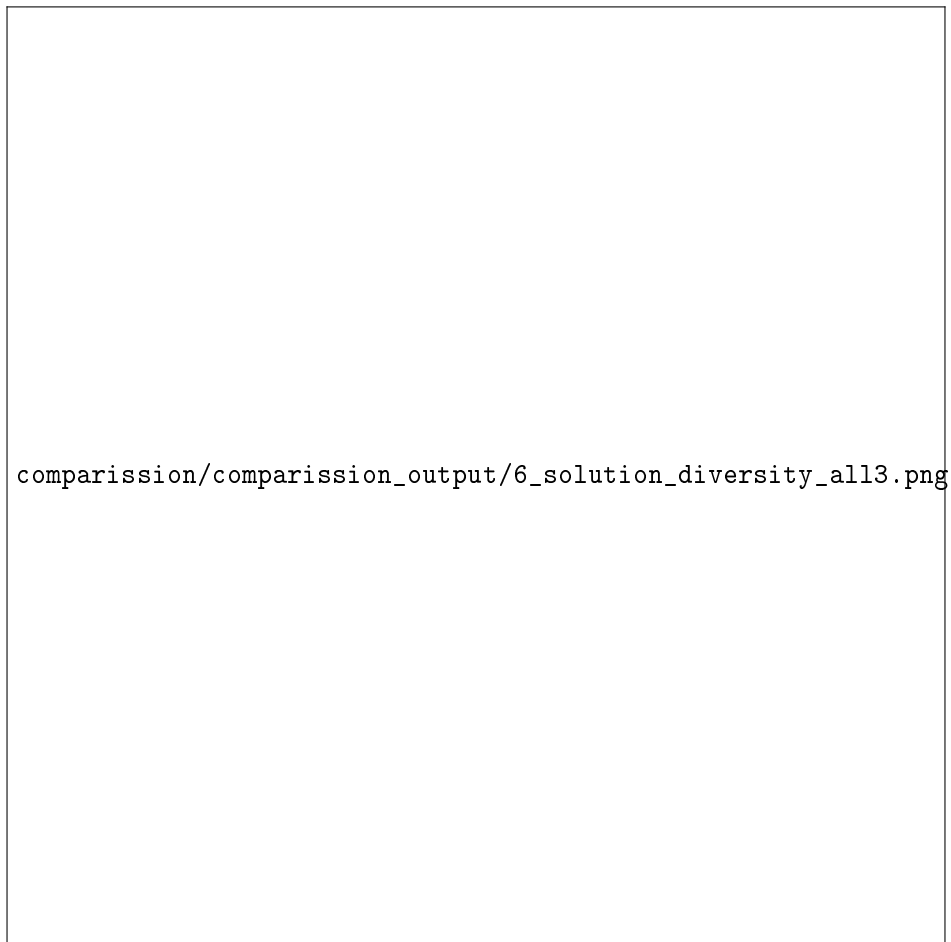


Figure 6.16: Solution diversity vs. quality (Pareto frontier) across DE, CMA-ES, and PPO.

6.5.7 Amortization Analysis (Multi-Board Generation Speed)

Figure ?? compares how total design time accumulates when generating layouts for 1 to 50 boards, using a pre-trained PPO model versus starting DE/CMA-ES from scratch for every board. Because CMA-ES and DE have no persistent memory of previously solved problems, they cannot save learned knowledge into a reusable model file and must re-run their entire mathematical search loop for every new board, so their cumulative time grows steeply, at roughly 1.5–2.5 s per additional layout. A pre-trained PPO model, in contrast, has its neural-network weights already saved to disk; it spends approximately 0.1 s reading the model file once, and thereafter generates each new layout almost instantly via direct inference, in under 0.02 s per layout.



Figure 6.17: Amortization analysis: cumulative design time for 1–50 boards, pre-trained PPO vs. DE/CMA-ES.

6.5.8 Constraint Violation Breakdown

Figure ?? decomposes residual penalty scores into a stacked bar chart of border overruns, overlaps, insert-clearance violations, and CoG imbalance, pinpointing each algorithm’s specific weaknesses – for instance, an algorithm may handle border bounds effortlessly while struggling with fine insert-clearance penalties, or vice versa.



Figure 6.18: Constraint violation breakdown by penalty category, across DE, CMA-ES, and PPO.

6.5.9 Discussion Summary

Table ?? synthesises the comparative results into a single qualitative summary across the three algorithms.

Table 6.4: Qualitative summary of algorithm trade-offs

Axis	DE	CMA-ES	PPO
Per-board time	Slowest ($\approx 2.5\text{s}$)	Faster ($\approx 1.5\text{s}$)	Fastest ($\approx 0.02\text{s}$, post-training)
Setup cost	None	None	High (training run, hours)
Scalability with N	$O(N^{1.9-2.2})$	$O(N^{1.9-2.2})$	$\approx O(N)$
Multi-board amortization	Poor (re-solves every time)	Poor (re-solves every time)	Excellent (one-time cost)
Placement bias	Boundary-hugging	Moderate	Interior-centred
Local-minima robustness	Prone to deadlocks	Strong (adaptive covariance)	Strong (learned policy)
Constraint compliance	Exact on accepted solutions	Exact on accepted solutions	Exact after independent re-verification

Chapter 7

Challenges and Solutions

Four major engineering challenges were encountered and resolved over the course of the project, summarised in Table ?? and elaborated below.

Table 7.1: Summary of engineering challenges and their resolutions

Challenge	Resolution
Restricted development environment	Prepared and pre-validated all algorithms, custom logic, and objective functions locally, offline, before transferring to the restricted target environment.
Speed vs. accuracy trade-off	Adopted Numba JIT compilation for all geometric constraint evaluation, enabling smooth execution even on modest hardware.
Zero-shot generalising PPO reward design	Carefully weighted relative penalty scaling so hard physical violations (overlap, border breach) always dominate the soft CoG term, preventing reward hacking.
DE local-minima stagnation	Introduced CMA-ES’s adaptive covariance-matrix step-size scaling to dynamically escape tight geometric deadlocks that plain DE could not resolve.

7.1 Developing Under Strict Environment Restrictions

The primary development environment had no internet access and disallowed installation of external software or additional Python libraries beyond the pre-installed Jupyter Notebook stack. This ruled out on-the-fly experimentation with new libraries within the target environment. The resolution was to prepare all key algorithms, custom penalty logic, and objective functions on a separate, internet-connected machine, thoroughly validate them there, and only then transfer the finalised code into the restricted environment for execution against the actual project data.

7.2 Balancing Speed versus Accuracy

Evaluating thousands of candidate component arrangements per second – checking for overlaps, board-boundary violations, and insert-hole spacing across every pairwise combination of components – is computationally expensive in standard interpreted Python, especially at the population sizes and generation counts required by DE (25×600) and CMA-ES (32 per generation), and at the environment-step throughput required by 8-way parallel PPO training. Numba JIT compilation was adopted to translate the core mathematical evaluation routines into optimized machine code, allowing calculations to run smoothly even on the modest hardware available (a MacBook with 18 GB RAM).

7.3 Designing a Zero-Shot Generalising Reward Function

Teaching the PPO agent to place components accurately across different panel sizes and component sets, without discovering degenerate shortcuts, proved to be the most delicate design challenge in the project. In particular, an under-constrained reward could allow the agent to trivially satisfy the CoG penalty term by collapsing every component toward the origin $(0, 0)$, while entirely ignoring the (also present, but insufficiently weighted) overlap penalty – a classic reward-hacking failure mode in RL. The resolution was to carefully weight relative penalty scaling so that hard physical violations (overlaps and border breaches) always dominate the soft CoG offset penalty, both in raw magnitude and after the log-compression described in Section ??, ensuring the agent could not improve its reward by sacrificing physical feasibility for CoG accuracy.

7.4 Fixing Algorithm Stagnation (Local-Minima Traps)

During development, the standard Differential Evolution solver was repeatedly observed to become stuck in local traps: configurations in which components were close to a fully valid arrangement, but could not move without transiently increasing the overlap penalty, which DE’s simple mutation/crossover mechanism could not reliably escape. This motivated the introduction of CMA-ES as a second solver: its adaptive covariance-matrix step-size scaling allows the search distribution to dynamically reshape itself around the local geometry of the problem, enabling it to escape tight geometric deadlocks that repeatedly stalled plain DE.

Chapter 8

Future Scope

Several directions for extending this work were identified over the course of the project:

- **Further PPO training.** Continuing training from the existing 3.5M-timestep checkpoint (no code changes required) would likely both lower the achievable CoG penalty further and raise the deterministic inference success rate beyond the current 2.75%.
- **Hyperparameter tuning pass** across PPO’s learning rate, rollout length (`n_steps`), entropy coefficient, action-annealing bounds (`MAX_STEP_MM/MIN_STEP_MM`), and the composite reward weighting (`SCORE_WEIGHTS`), to further improve CG penalty or inference success rate.
- **Tuning the number of returned layouts and diversity threshold** (`N_EPISODES`, `DEDUP_DIST_MM`) to give engineers more than five candidate layouts, or a stricter/looser notion of what counts as a genuinely “distinct” alternative.
- **Component rotation.** Extending the design-variable vector to include a discrete or continuous orientation angle per component, rather than treating footprints as axis-fixed.
- **Field of View (FoV) keepout cones.** Extending the state space to include 3D volumetric clearance cones for optical sensors, star trackers, and cameras, ensuring that no adjacent component obstructs their required angular sightlines.
- **Thermal dissipation and hot-spot spacing.** Incorporating thermal dissipation models as spatial constraints to maintain minimum separation distances between high-power units (e.g., PDUs) and heat-sensitive components, thereby preventing the formation of thermal hot-spots.
- **Connector mating and harness bend radii.** Adding directional clearance envelopes in front of connector faces to account for cable routing, connector mating depth, and the minimum allowable wire-harness bend radii.
- **Thermal and structural co-optimization.** Incorporating thermal dissipation paths and structural load paths as additional objective/constraint terms alongside the current CoG-centric formulation.
- **Hybrid solvers.** Investigating a PPO-global-search-plus-CMA-ES-polish hybrid (considered but not selected during initial design, Table ??), which could combine PPO’s amortized speed with CMA-ES’s precise local refinement.

Chapter 9

Conclusion

This project delivered a complete, JSON-configuration-driven pipeline for automating dual-sided spacecraft panel component placement under a realistic hierarchy of physical constraints: exact Centre-of-Gravity balancing, zero-overlap same-face clearance (including directional Connector-Face overrides), panel-border containment, and cross-face insert-hole minimum-separation rules. Starting from a single-cell, guide-supplied Differential Evolution prototype capable only of CoG balancing on a handful of interactively entered components, the project progressively layered in every required constraint, generalised to circular components and true front/back dual-sided placement, and ultimately implemented and rigorously compared three qualitatively distinct optimization paradigms – Differential Evolution, CMA-ES, and Proximal Policy Optimization – all sharing a common Numba-JIT-accelerated constraint core and a common convention of generating five distinct, fully compliant layouts per run.

The comparative study demonstrated that while DE and CMA-ES remain simple, reliable, per-board solvers – with CMA-ES’s adaptive covariance-matrix mechanism providing measurably better robustness against local-minima stagnation than plain DE – both must always resolve the placement problem from scratch, with wall-clock cost per board of roughly 1.5–2.5 s and super-linear scaling with the number of components. The trained PPO policy, in contrast, amortizes a one-time training cost into sub-0.02-second, near-linear-scaling inference for every subsequent board, while still satisfying every hard geometric constraint exactly after independent re-verification, making it especially attractive for a design workflow that must repeatedly generate layouts for many similar panel variants. The successful, unmodified application of all three solvers to a new, previously unseen panel specification supplied late in the project confirms that the resulting pipeline generalises beyond its original development configuration, fulfilling the project’s central objective of giving mechanical design engineers at the Mechanical Integration Division an automated, multi-alternative tool for spacecraft component placement in place of a slow, manual, single-alternative process.

Bibliography

- [1] Hansen, N. (2016). *The CMA Evolution Strategy: A Tutorial*. arXiv preprint arXiv:1604.00772.
- [2] Storn, R., & Price, K. (1997). Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. *Journal of Global Optimization*, 11(4), 341–359.
- [3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
- [4] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [5] Sechen, C., & Sangiovanni-Vincentelli, A. (1985). The TimberWolf Placement and Routing Package. *IEEE Journal of Solid-State Circuits*, 20(2), 510–522.
- [6] Mirhoseini, A., et al. (2021). A Graph Placement Methodology for Fast Chip Design. *Nature*, 594(7862), 207–212.
- [7] Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A JIT Compiler for Python-Driven Array-Oriented Scientific Computing. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, 1–6.
- [8] Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations in Python. *Journal of Machine Learning Research*, 22(268), 1–8.

Appendix A

Selected Code Listings

A.1 DE Solver Invocation and Post-Filtering

```
1 result = differential_evolution(  
2     evaluate, bounds,  
3     args=(masses, lengths, widths, clearance, req_d_matrix,  
4         flat_offsets, offset_counts, offset_indices,  
5         is_back_side, clearance_dirs,  
6         panel_width, panel_height, border_spacing),  
7     maxiter=600, popsize=25, tol=1e-6,  
8     strategy='best1bin', seed=seed,  
9     workers=-1, updating='deferred'  
10 )  
11  
12 # STRICT FILTER: Reject layout solutions containing non-zero  
13 # penalty scores or numerical anomalies  
14 if result.fun > 1e-3 or np.any(np.isinf(result.x)) or np.any(np.isnan(result.x)):  
15     continue  
16  
17 cx_v = result.x[:, :2]  
18 cy_v = result.x[:, 1:2]
```

Listing A.1: Full DE solver invocation and strict-compliance post-filter (DE/de.py)

A.2 CMA-ES Solver Configuration

```
1 sigma0 = 15.0 # Initial search step size (standard deviation)  
2  
3 cma_opts = {  
4     'popsize': 32, # Candidate solutions generated per generation  
5     # ... bounds, tolerance, and seed options ...  
6 }  
7  
8 es = cma.CMAEvolutionStrategy(x0, sigma0, cma_opts)
```

Listing A.2: CMA-ES strategy initialisation (CMA-ES/cma-es-local.py)

A.3 PPO Reward Computation (conceptual)

```
1 composite_score = (  
2     3.0 * np.log1p(border_penalty) +  
3     3.0 * np.log1p(overlap_penalty) +  
4     10.0 * np.log1p(insert_penalty) +  
5     0.5 * np.log1p(cg_penalty)  
6 )  
7  
8 reward = (prev_composite_score - composite_score) - STEP_COST  
9 if layout_is_fully_valid:  
10     reward += SUCCESS_BONUS  
11     terminated = True
```

Listing A.3: Composite, log-compressed PPO reward formulation (PPO training notebook)

A.4 PPO Command-Line Inference Interface

```
1 python gen_from_inference.py \  
2     --json layout1.json \  
3     --model_dir optimized_layouts_rl \  
4     --output_dir ppo_output \  
5     --episodes 800 \  
6     --parallel 8 \  
7     --max_layouts 5 \  
8     --dedup_dist 25.0 \  
9     --deterministic \  
10    --seed 42
```

Listing A.4: Standalone multi-layout generation CLI (PPO/gen_from_inference.py)

Appendix B

JSON Configuration Schema (Full)

The complete board and component configuration schema, of which Listing ?? in Chapter ?? is an excerpt, is contained in the project's `layout1.json` (baseline configuration) and `layout2.json` (new-layout case study, Chapter ??) files, each declaring a **BOARD** object (length, breadth, weight, border, and clearance in millimetres/kilograms) and a **COMPONENTS** object split into **FRONT** and **BACK** arrays, with each component entry specifying its name, geometry (rectangular length/breadth or circular diameter), mass, insert diameter and quantity, an optional quantity multiplier for repeated identical parts, and an optional directional Connector-Face clearance override, exactly as described in Section ??.